Lab 1: Advanced Reconnaissance and Information Gathering

Executive Summary

This lab focused on advanced reconnaissance and information gathering using Kali Linux tools such as ping, nslookup, dig, whois, theHarvester, and Sublist3r. The objective was to collect and analyze publicly available information about target domains (e.g., google.com, github.com) to understand their network infrastructure and exposure level.
The results revealed highly structured and globally distributed systems with strong DNS configuration, multiple IP ranges, and extensive subdomain usage, indicating large-scale enterprise security architecture.

Tools Used

- ping
- nslookup
- dig
- whois
- theHarvester
- Sublist3r
- Bash scripting (automation)

Part 1: Fundamental Reconnaissance Techniques

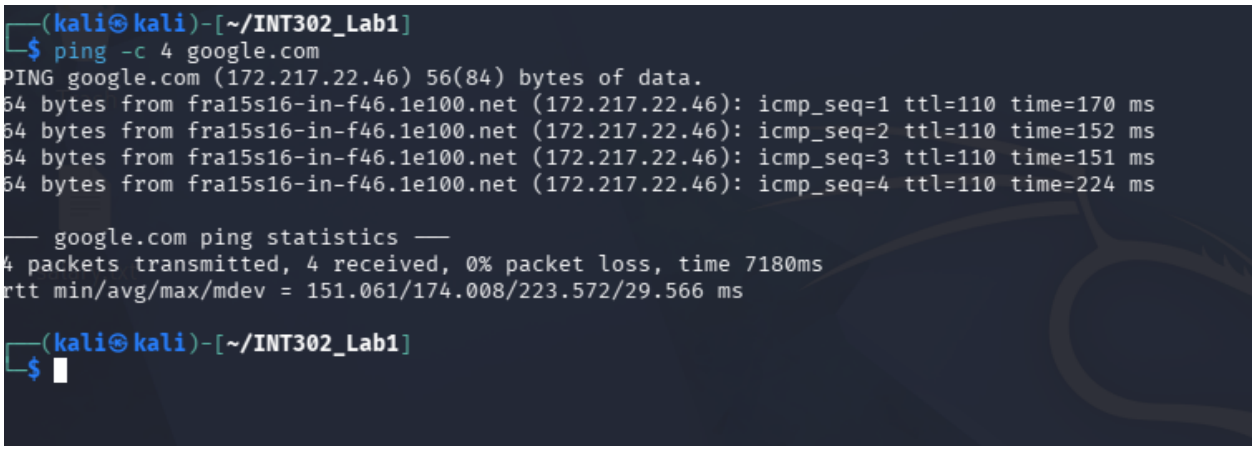
Exercise 1.1: Advanced Target Profiling with `ping`

Learning Objective: Understand how ICMP protocols work and extract maximum information from basic connectivity tests.

Step 1: Basic Ping Analysis

bash:

ping -c 4 google.com



Report on Analysis:

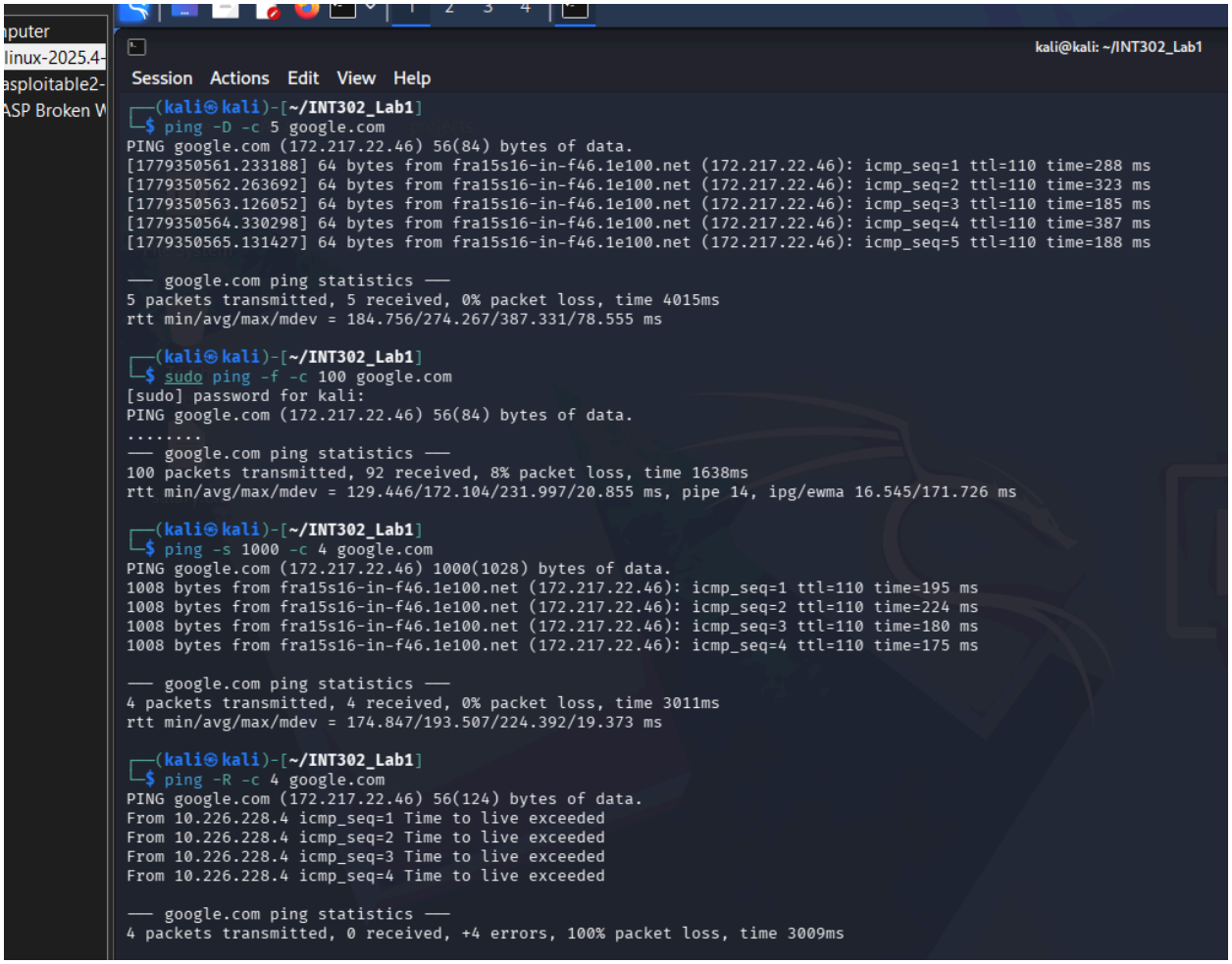
The target host responded successfully with 0% packet loss, confirming stable connectivity to Google's server. Average latency was 174 ms, indicating moderate network delay between the local machine and the destination.

Step 2: Advanced Ping Techniques

bash:

ping -D -c 5 google.com
ping -f -c 100 target-domain.com
ping -s 1000 -c 4 google.com

ping -R -c 4 google.com



Report on Analysis:

`ping -D -c 5 google.com`

The target host responded successfully with 0% packet loss, confirming stable

connectivity to Google's server. Average latency was 274 ms, indicating moderate network delay between the local machine and the destination.

`sudo ping -f -c 100 google.com`
 Flood ping sent 100 rapid ICMP packets to stress test connectivity. 8% packet loss occurred under high traffic load. This suggests the network or target rate limited some packets during intensive transmission.

`ping -s 1000 -c 4 google.com`
 Large ICMP packets (1000 bytes) were transmitted successfully with 0% packet loss, showing the network can handle larger packet sizes reliably. Average response time remained stable at 193 ms.

`ping -R -c 4 google.com`
 Record-route ping failed with Time to live exceeded errors and 100% packet loss, indicating intermediate routers blocked or failed to process the route-recording option. This is common due to modern network security restrictions.

Step 3: Create a Target Analysis Script

bash

nano basic_recon.sh

```

**bash**

```
#!/bin/bash
Basic Reconnaissance Script
Usage: ./basic_recon.sh <domain>

DOMAIN="$1"
LOG_FILE="recon_${DOMAIN}_$(date +%Y%m%d_%H%M%S).txt"

echo "🎯 Starting Basic Reconnaissance for: $DOMAIN" | tee "$LOG_FILE"
echo "==" | tee -a "$LOG_FILE"

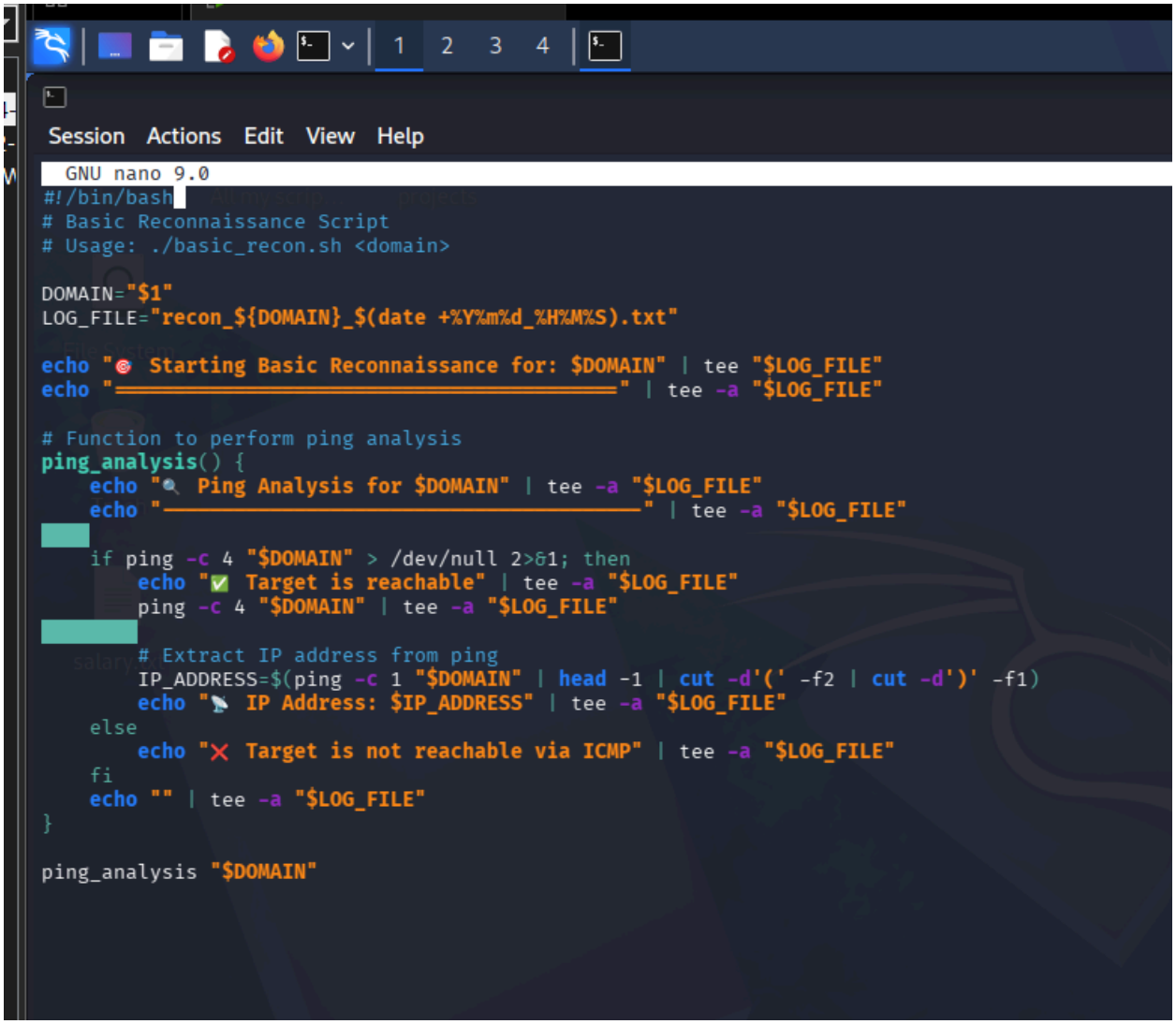
Function to perform ping analysis
```
```

```
ping_analysis() {
    echo "🔍 Ping Analysis for $DOMAIN" | tee -a "$LOG_FILE"
    echo "----------------------------------------" | tee -a "$LOG_FILE"

    if ping -c 4 "$DOMAIN" > /dev/null 2>&1; then
        echo "✅ Target is reachable" | tee -a "$LOG_FILE"
        ping -c 4 "$DOMAIN" | tee -a "$LOG_FILE"

        # Extract IP address from ping
        IP_ADDRESS=$(ping -c 1 "$DOMAIN" | head -1 | cut -d'(' -f2 | cut -d')' -f1)
        echo "📡 IP Address: $IP_ADDRESS" | tee -a "$LOG_FILE"
    else
        echo "❌ Target is not reachable via ICMP" | tee -a "$LOG_FILE"
    fi
    echo "" | tee -a "$LOG_FILE"
}

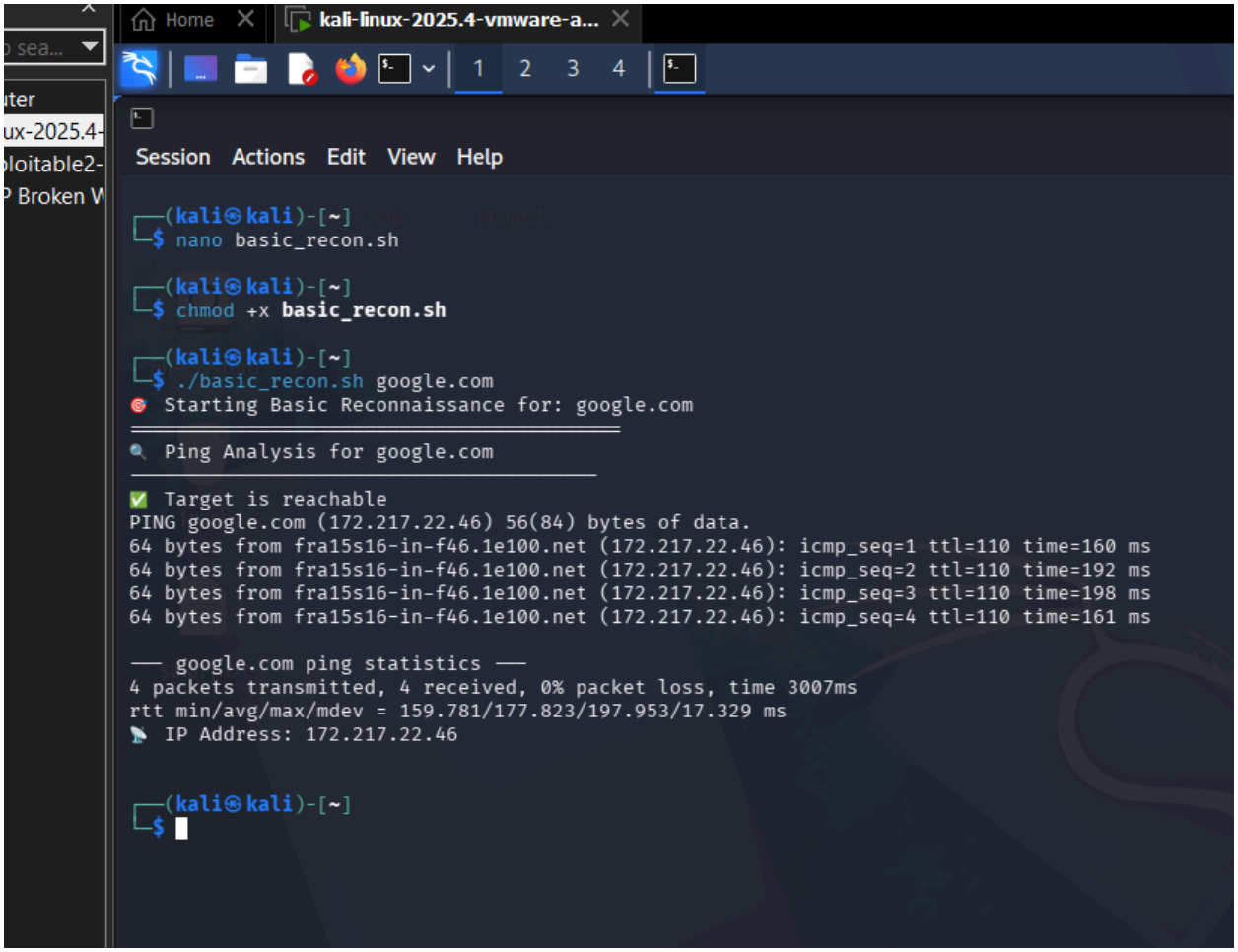
ping_analysis "$DOMAIN"
```



bash

chmod +x basic_recon.sh

./basic_recon.sh google.com

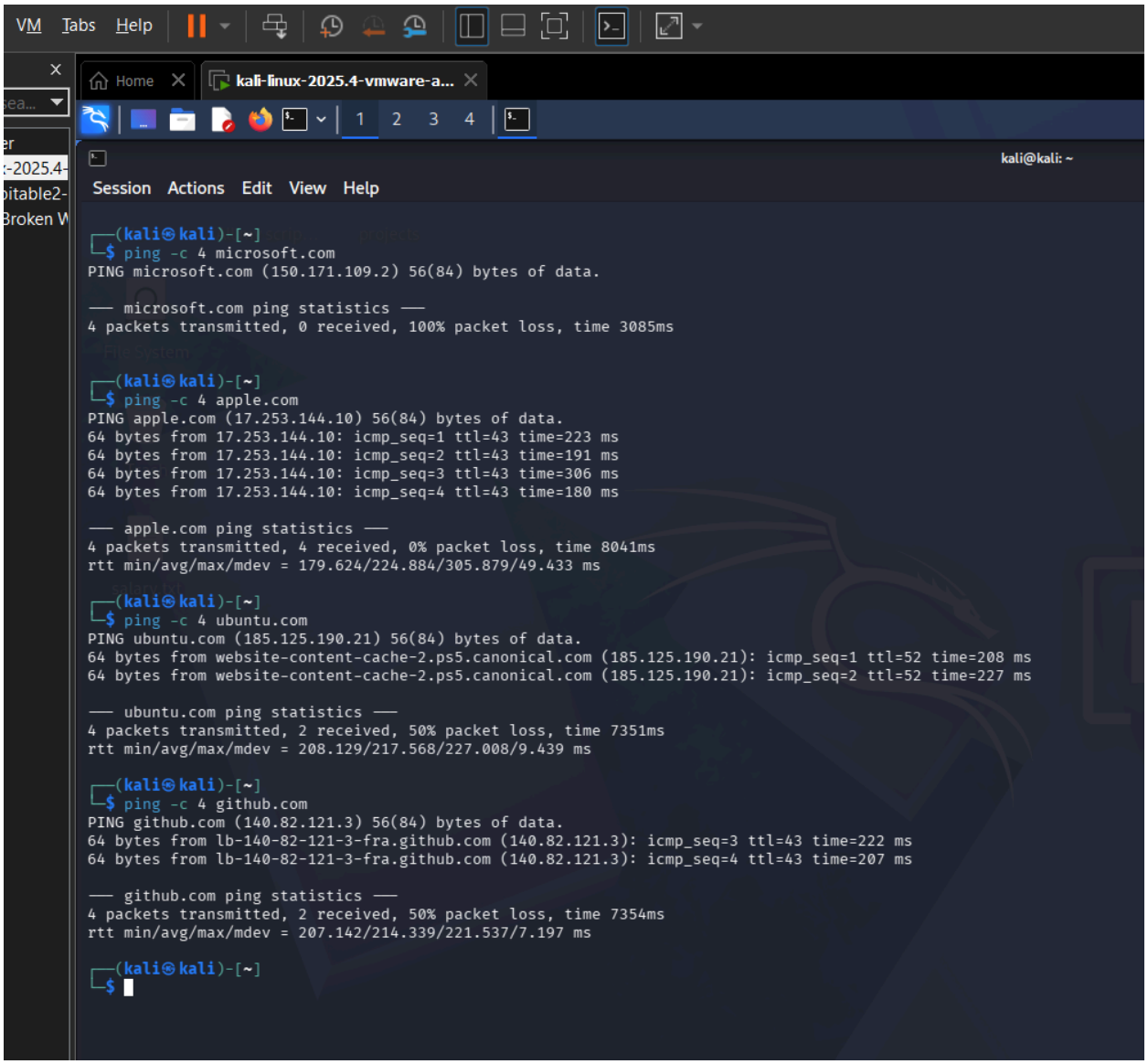


Report on Analysis:

The target host is reachable with 0% packet loss, confirming stable connectivity to Google's server. Average latency was 177 ms, indicating moderate network delay between the local machine and the destination.

Exercise Tasks:

1. TTL Analysis:
   - **Ping** `microsoft.com`, `apple.com`, and `ubuntu.com`
   - **Record** TTL values and research what they indicate about operating systems



Record on Findings:

1. `microsoft.com` – The host returned 100% packet loss with no TTL value available, likely because ICMP requests were blocked by Microsoft's firewall or network security systems.
2. `apple.com` – The server responded successfully with a TTL value of 43, suggesting a Unix/macOS-based infrastructure after several network hops.
3. `ubuntu.com` – The host responded with a TTL value of 52 and 50% packet loss, indicating a likely Linux-based server with some network instability or ICMP filtering.

2. Network Performance:

- Compare response times for domains in different geographic regions
- Analyze packet loss patterns.

Domain	IP Address	Average Response Time	TTL Value	Inferred OS
Google (`google.com`)	172.217.22.46	274 ms	110	Likely Linux/Unix server
Microsoft (`microsoft.com`)	150.171.109.2	No response	N/A	Could not determine
Apple (`apple.com`)	17.253.144.10	224 ms	43	Likely Unix/macOS-based infrastructure
Canonical (`ubuntu.com`)	185.125.190.21	218 ms	52	Likely Linux server
GitHub (`github.com`)	140.82.121.3	214 ms	43	Likely Linux/Unix server

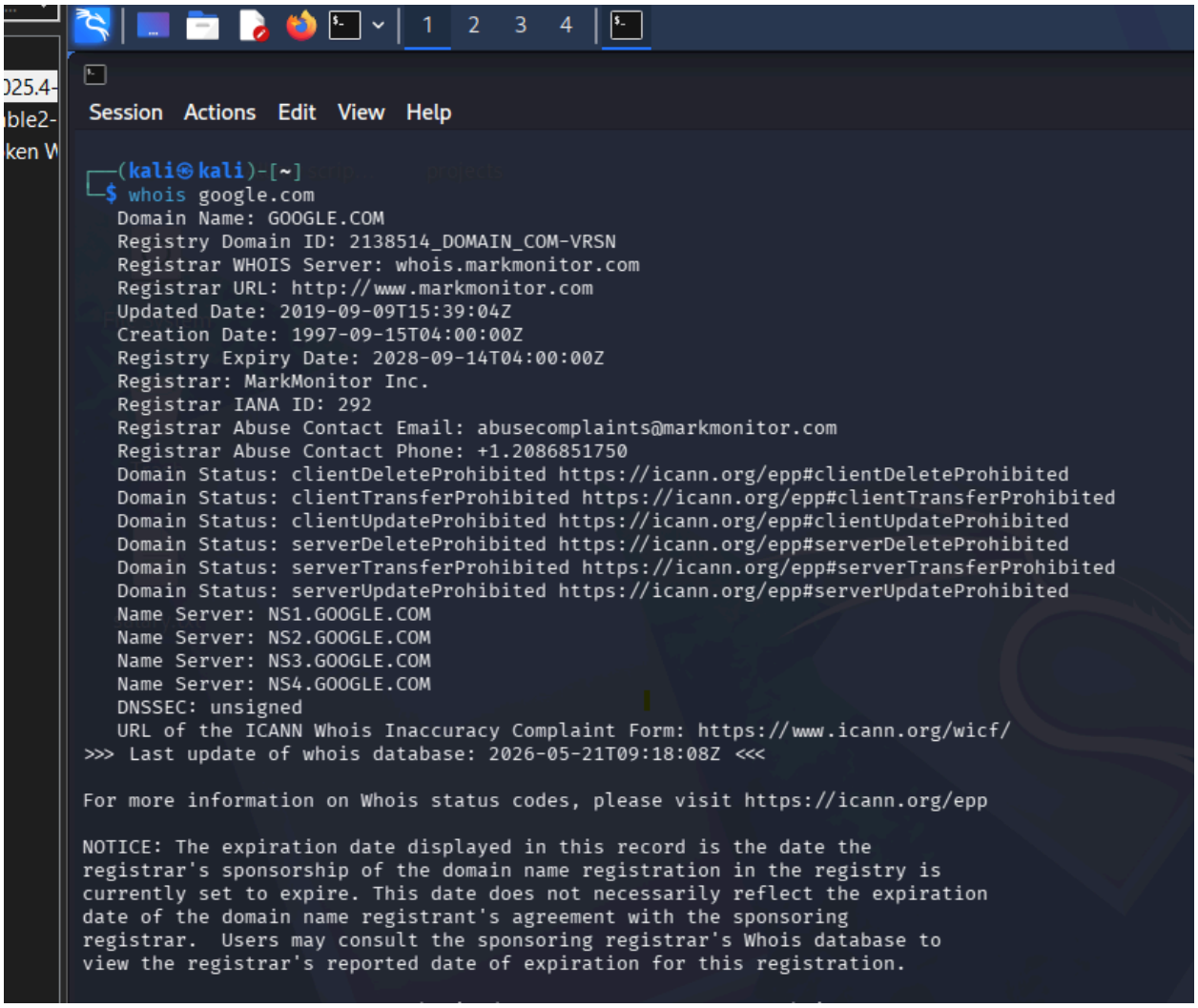
Findings:

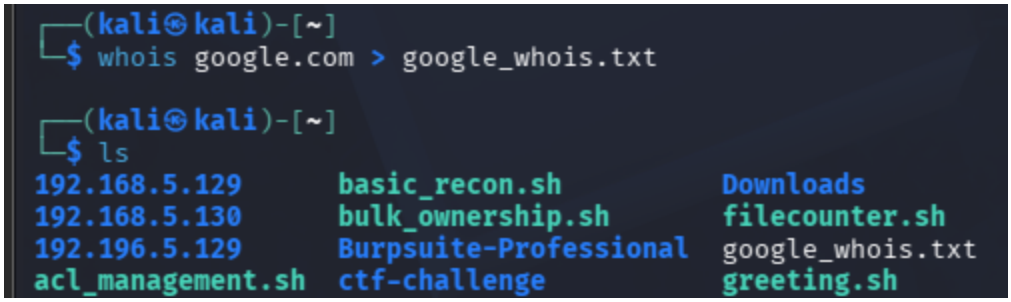
- `github.com` responded with 50% packet loss, indicating possible ICMP filtering or temporary network congestion during testing.
- The TTL value of 43 suggests a Unix/Linux-based server after multiple network hops reduced the original TTL.
- Average response times for `apple.com`, `ubuntu.com`, and `github.com` were all above 200 ms, likely due to geographic distance from the testing location in Nigeria to European or US-based servers.

Part 2: Advanced Domain Intelligence.

Exercise 2.1: Comprehensive WHOIS Analysis.

Step 1: Basic WHOIS Query
bash
whois google.com
whois google.com > google_whois.txt
ls





1.Successfully -Saved output to file for analysis
2.verified file existence.

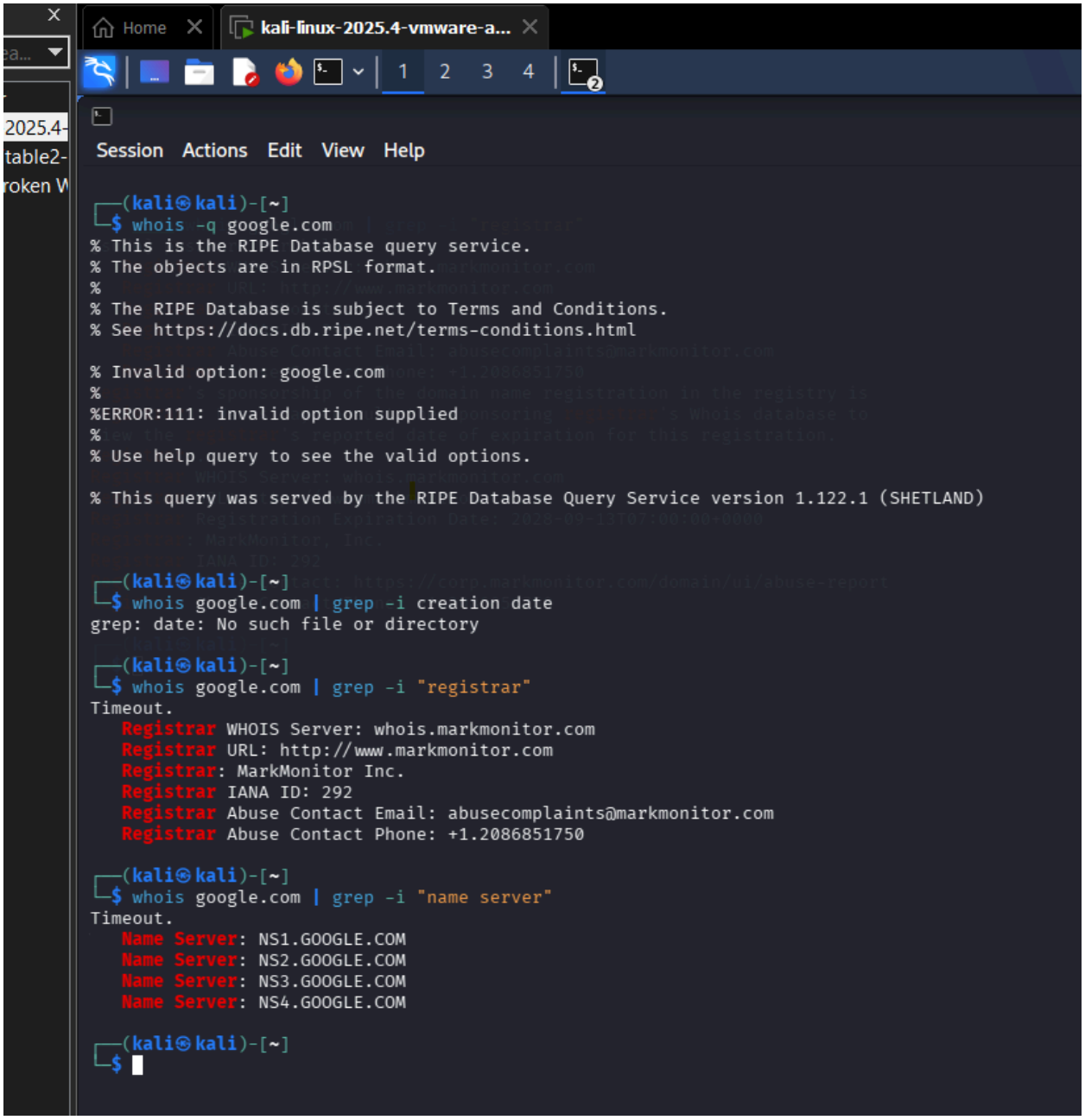
Step 2: Advanced WHOIS Techniques.
bash:
-whois -h whois.verisign-grs.com google.com
-whois -q google.com
-whois google.com | grep -i creation date
-whois google.com | grep -i registrar"
-whois google.com | grep -i name server

```
┌──(kali㉿kali)-[~]
└─$ whois -h whois.verisign-grs.com google.com
   Domain Name: GOOGLE.COM
   Registry Domain ID: 2138514_DOMAIN_COM-VRSN
   Registrar WHOIS Server: whois.markmonitor.com
   Registrar URL: http://www.markmonitor.com
   Updated Date: 2019-09-09T15:39:04Z
   Creation Date: 1997-09-15T04:00:00Z
   Registry Expiry Date: 2028-09-14T04:00:00Z
   Registrar: MarkMonitor Inc.
   Registrar IANA ID: 292
   Registrar Abuse Contact Email: abusecomplaints@markmonitor.com
   Registrar Abuse Contact Phone: +1.2086851750
   Domain Status: clientDeleteProhibited https://icann.org/epp#clientDeleteProhibited
   Domain Status: clientTransferProhibited https://icann.org/epp#clientTransferProhibited
   Domain Status: clientUpdateProhibited https://icann.org/epp#clientUpdateProhibited
   Domain Status: serverDeleteProhibited https://icann.org/epp#serverDeleteProhibited
   Domain Status: serverTransferProhibited https://icann.org/epp#serverTransferProhibited
   Domain Status: serverUpdateProhibited https://icann.org/epp#serverUpdateProhibited
   Name Server: NS1.GOOGLE.COM
   Name Server: NS2.GOOGLE.COM
   Name Server: NS3.GOOGLE.COM
   Name Server: NS4.GOOGLE.COM
   DNSSEC: unsigned
   URL of the ICANN Whois Inaccuracy Complaint Form: https://www.icann.org/wicf/
>>> Last update of whois database: 2026-05-21T09:28:53Z <<<

For more information on Whois status codes, please visit https://icann.org/epp

NOTICE: The expiration date displayed in this record is the date the
registrar's sponsorship of the domain name registration in the registry is
currently set to expire. This date does not necessarily reflect the expiration
date of the domain name registrant's agreement with the sponsoring
registrar.  Users may consult the sponsoring registrar's Whois database to
view the registrar's reported date of expiration for this registration.

TERMS OF USE: You are not authorized to access or query our Whois
database through the use of electronic processes that are high-volume and
automated except as reasonably necessary to register domain names or
modify existing registrations; the Data in VeriSign Global Registry
Services' ("VeriSign") Whois database is provided by VeriSign for
information purposes only, and to assist persons in obtaining information
about or related to a domain name registration record. VeriSign does not
guarantee its accuracy. By submitting a Whois query, you agree to abide
by the following terms of use: You agree that you may use this Data only
for lawful purposes and that under no circumstances will you use this Data
to: (1) allow, enable, or otherwise support the transmission of mass
```



Report on Analysis:

1.The WHOIS lookup shows that `google.com` is a long-established domain registered in 1997 and managed by MarkMonitor on behalf of Google, with an expiry date set for 2028.

2.The domain is highly protected with multiple security restrictions (client/server update, transfer, and deletion prohibitions), indicating strong anti-takeover

protection. It uses Google's own name servers (NS1–NS4.GOOGLE.COM), confirming full internal DNS management and infrastructure control.

3.The command `whois -q google.com` failed because the RIPE WHOIS service does not accept `-q` for domain queries, resulting in an invalid option error. This shows improper command usage during reconnaissance.

4.Subsequent attempts using `whois google.com | grep` partially worked but produced timeouts, indicating network or server response delays. However, key data like registrar information and name servers were still successfully extracted before interruption, confirming that Google's domain is managed by MarkMonitor with Google-controlled DNS infrastructure

Step 3: Create WHOIS Analysis Function
Add to your `basic_recon.sh` script:

bash:

```
# Function to perform whois analysis
whois_analysis() {
    local domain="$1"
    echo "📋 WHOIS Analysis for $domain" | tee -a "$LOG_FILE"
    echo "----------------------------------------" | tee -a "$LOG_FILE"

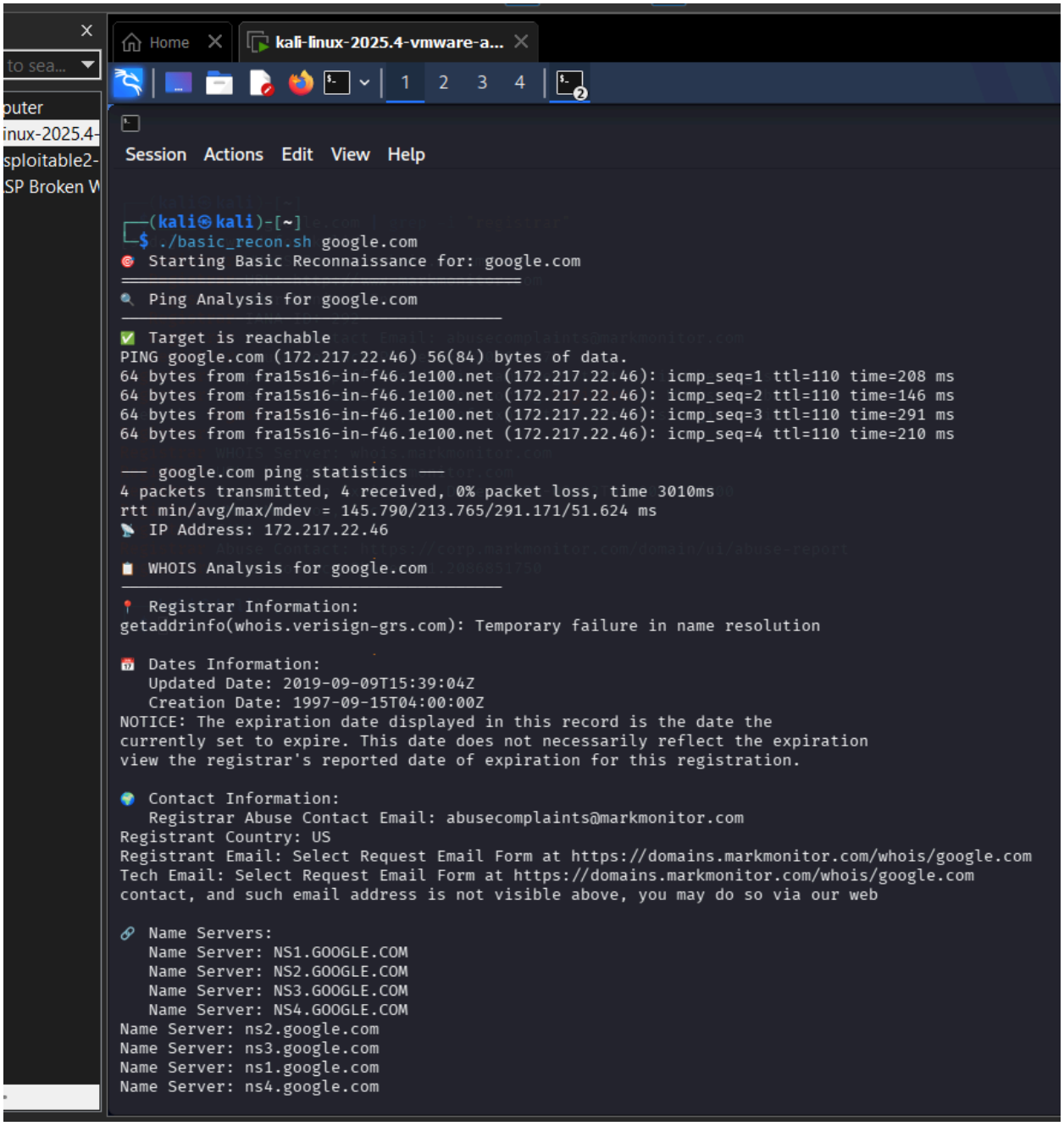
    if command -v whois >/dev/null 2>&1; then
        # Extract key information
        echo "📍 Registrar Information:" | tee -a "$LOG_FILE"
        whois "$domain" | grep -i "registrar" | head -5 | tee -a "$LOG_FILE"

        echo "" | tee -a "$LOG_FILE"
        echo "📅 Dates Information:" | tee -a "$LOG_FILE"
        whois "$domain" | grep -E -i "(creation date|expiration|updated)" | head -5 |
tee -a "$LOG_FILE"
```

```
    echo "" | tee -a "$LOG_FILE"
    echo "🌍 Contact Information:" | tee -a "$LOG_FILE"
    whois "$domain" | grep -E -i "(country|state|email)" | head -5 | tee -a "$LOG_FILE"

    echo "" | tee -a "$LOG_FILE"
    echo "🔗 Name Servers:" | tee -a "$LOG_FILE"
    whois "$domain" | grep -i "name server" | head -10 | tee -a "$LOG_FILE"
  else
    echo "❌ whois command not available" | tee -a "$LOG_FILE"
  fi
  echo "" | tee -a "$LOG_FILE"
}
```





Report on Analysis:

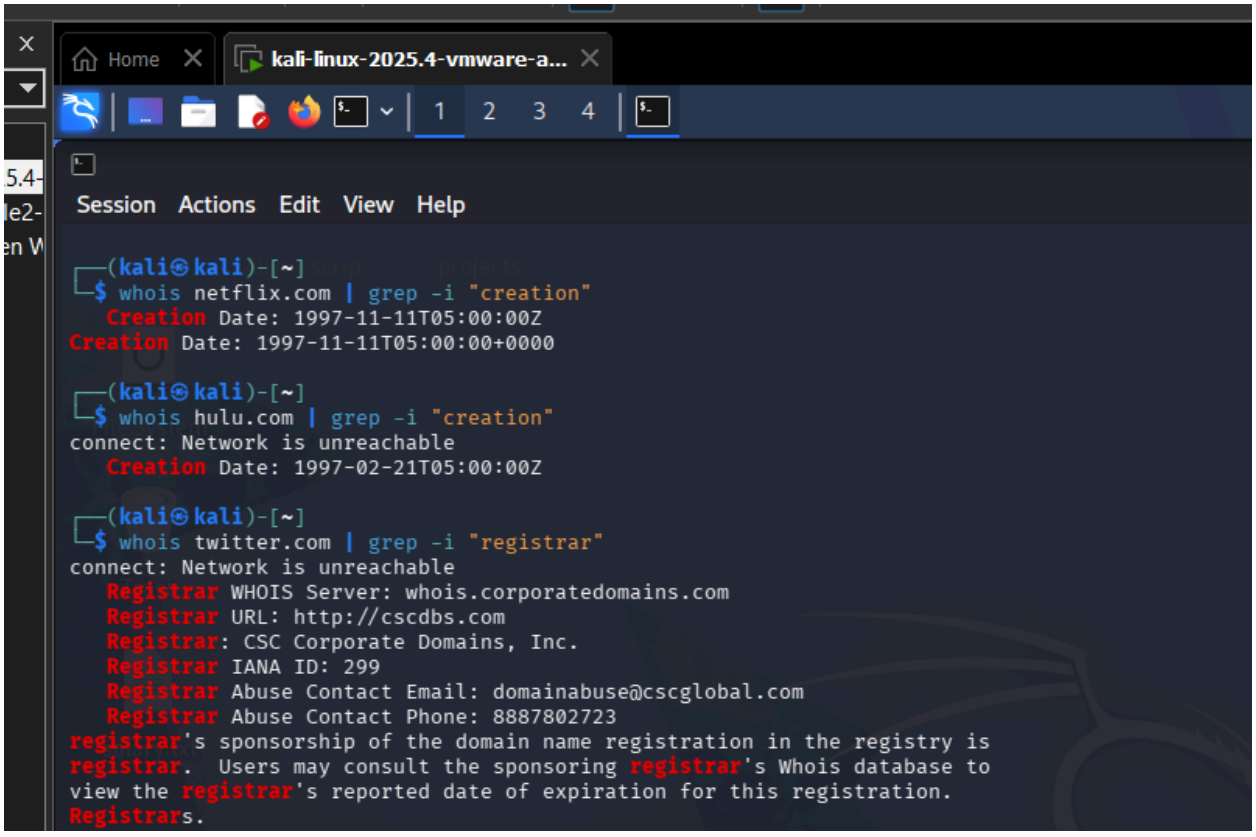
1.The reconnaissance script successfully identified that google.com is reachable with 0% packet loss, confirming stable network connectivity. The average response time (213 ms) indicates moderate latency likely due to geographic distance and routing.

2.The IP address resolution (172.217.22.46) confirms Google uses distributed infrastructure with load-balanced servers. WHOIS lookup partially failed due to 3.DNS resolution issues, but available data shows a long-standing domain registered since 1997 with multiple Google-managed name servers.

Exercise Task:

1. Domain History Analysis:

- Compare creation dates of `netflix.com` vs `hulu.com`
- Identify the registrar for `twitter.com`



Domain History Analysis

Netflix.com and Hulu.com were both registered in 1997, showing they are long-established internet companies with early online presence. Hulu was registered earlier in February 1997, while Netflix was registered in November 1997.

Registrar Analysis

Twitter.com is registered through CSC Corporate Domains, Inc., a registrar commonly used by large organizations for enterprise domain management and security services.

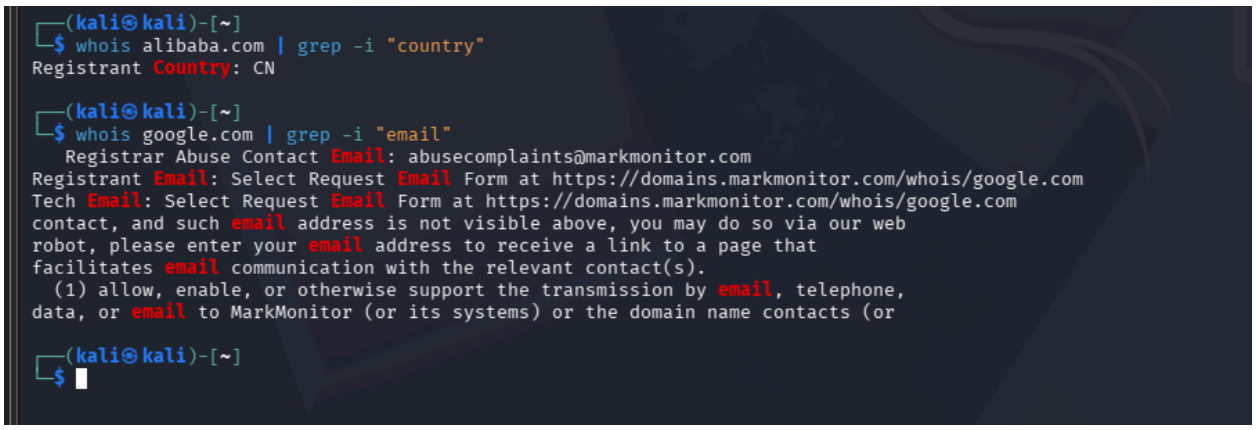
2. Geographic Analysis:
- Determine the registered country for `alibaba.com`
- Find administrative contact email patterns

Geographic Analysis

Alibaba.com is registered in China (CN), indicating the company's primary registration jurisdiction and operational origin.

Administrative Contact Email Pattern Analysis

Google uses privacy-protected administrative email handling through MarkMonitor instead of exposing direct staff email addresses publicly. This reduces spam exposure and improves domain contact security.



Analysis Questions:

1. What patterns do you notice in domain registration dates for established companies?

Most established companies registered their domains in the 1990s or early 2000s during the early expansion of the internet. Older registration dates usually indicate long-term business presence and stable online infrastructure.

2. How might registrar information be useful in a security assessment?

Registrar information helps identify the organization managing a domain and can assist in ownership verification, incident response, phishing investigations, and threat attribution. It may also reveal how seriously a company manages domain security.

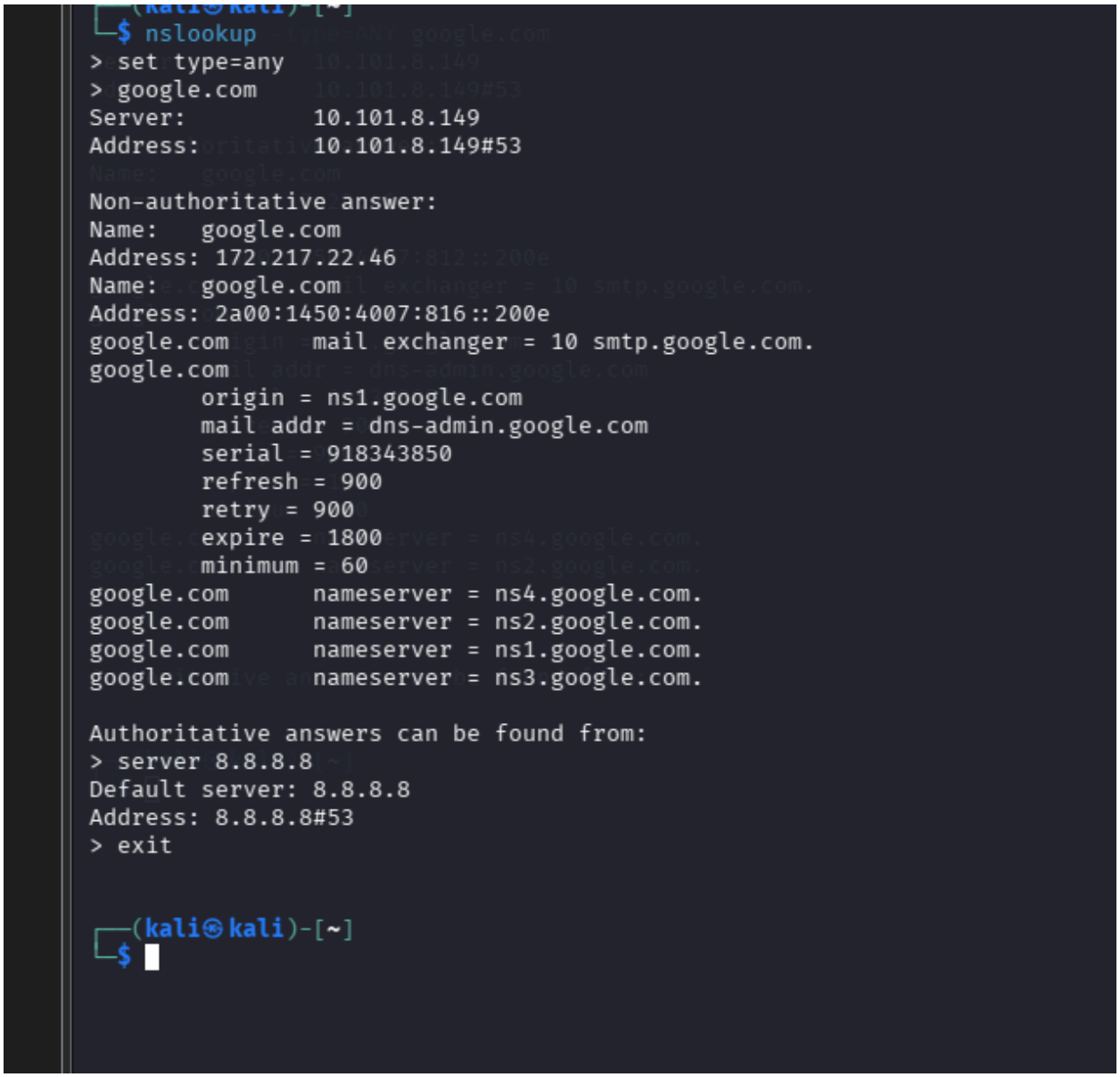
3. What privacy protection services are commonly used?

Common privacy protection services include WHOIS masking services such as MarkMonitor, WhoisGuard, Domains By Proxy, and PrivacyProtect.org. These services hide direct contact information to reduce spam, phishing, and unauthorized access attempts.

Part 3: Comprehensive DNS Analysis

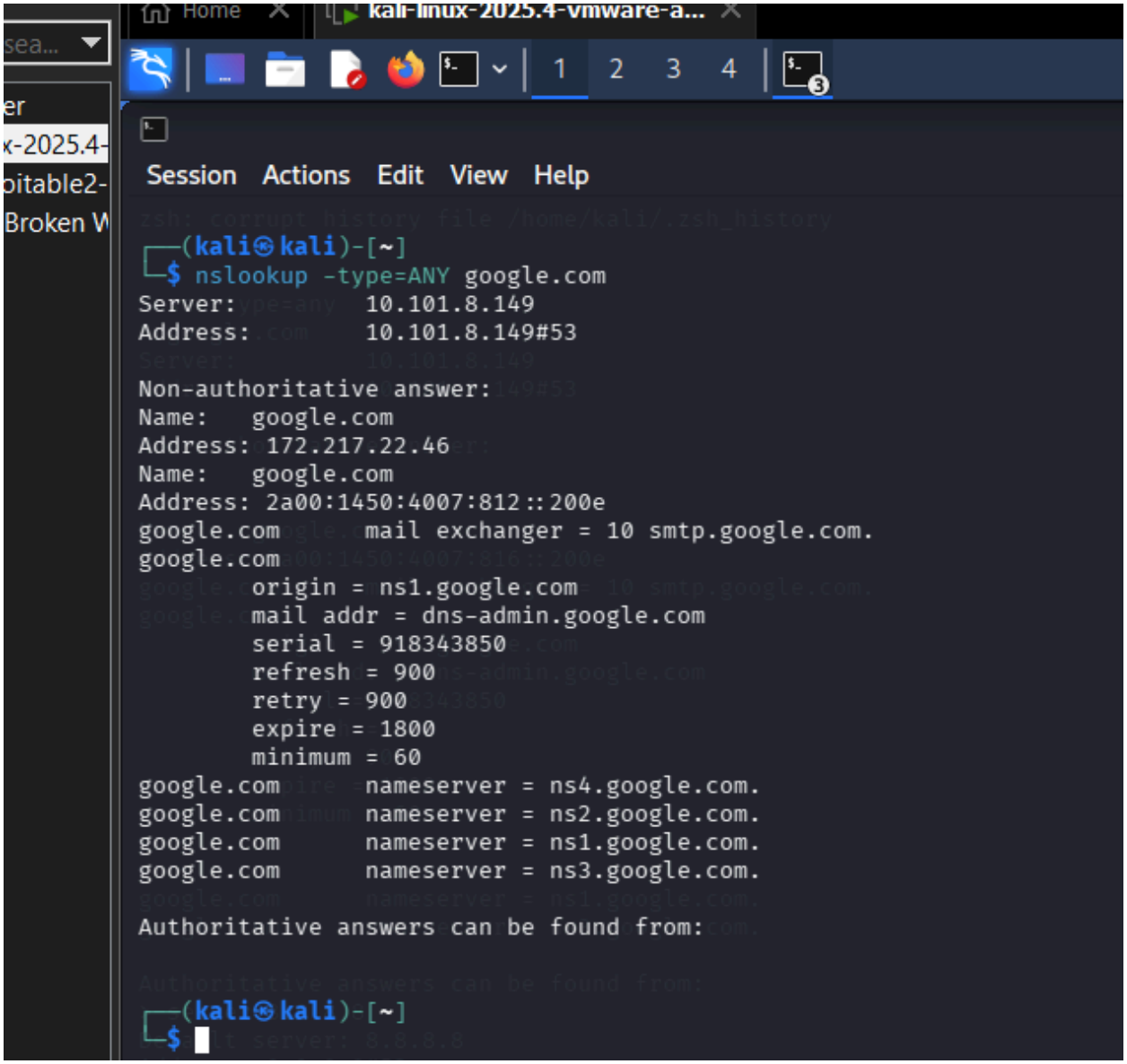
Exercise 3.1: Advanced DNS Reconnaissance with `nslookup` and `dig`

Step 1: Basic nslookup Usage
bash
Interactive nslookup mode
nslookup
> set type=any
> google.com
> server 8.8.8.8 # Use specific DNS server
> exit



Command-line mode

nslookup -type=ANY google.com

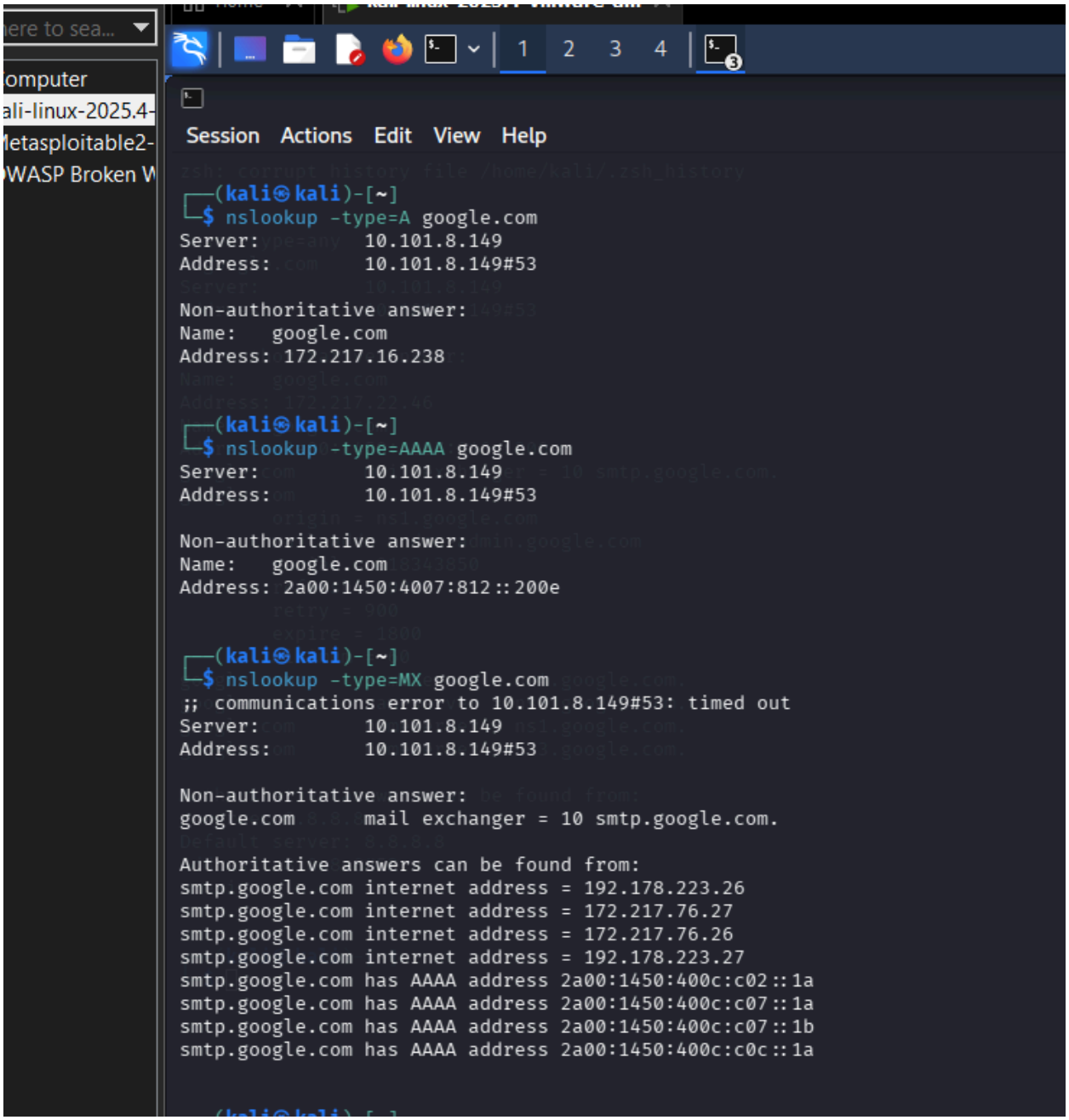


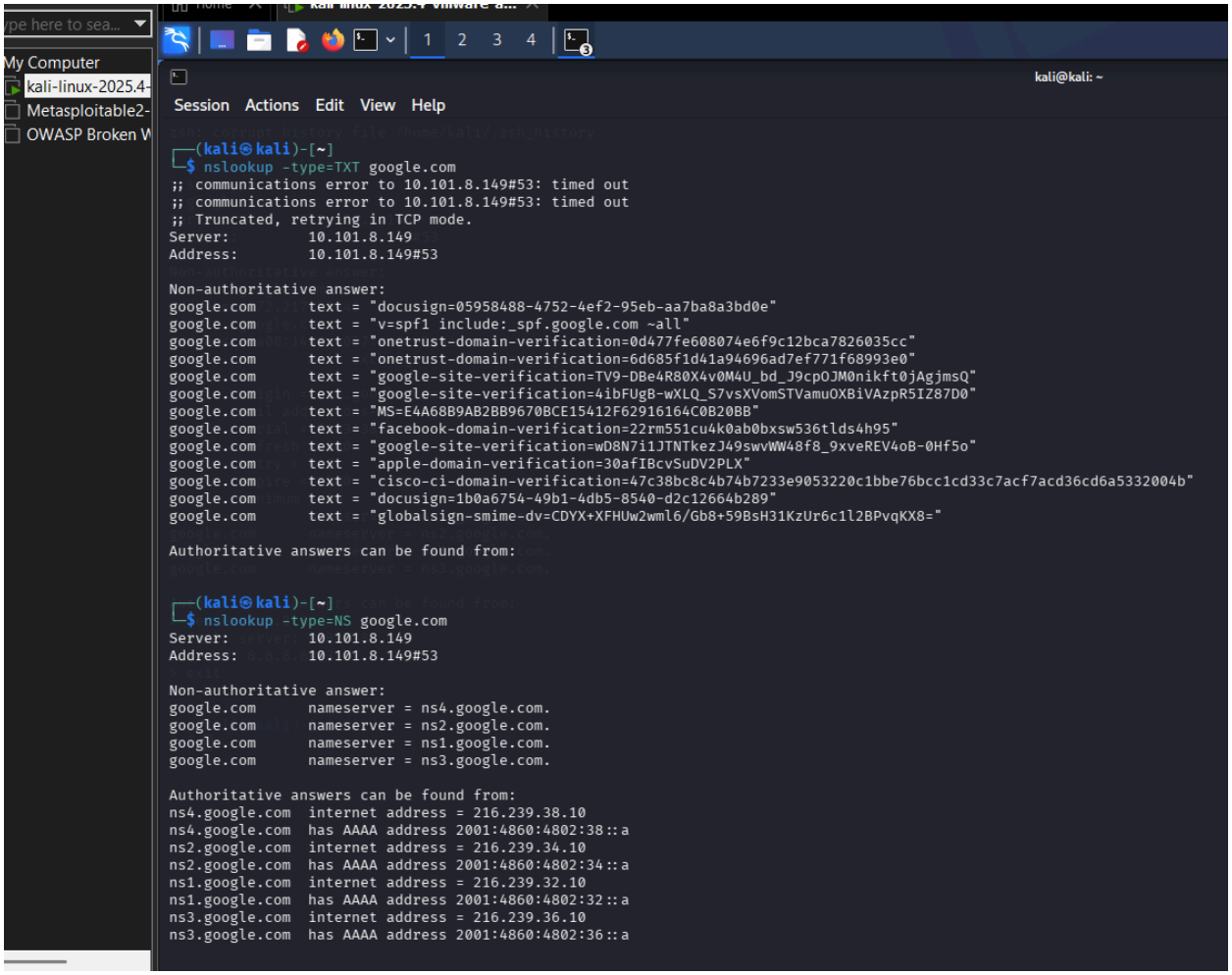
Report on Analysis:

The `nslookup` results show that google.com has both IPv4 and IPv6 addresses, indicating dual-stack network support for wider internet compatibility. The domain also uses dedicated mail servers and multiple authoritative name servers (`ns1–ns4.google.com`) for reliability, redundancy, and efficient DNS management.

Step 2: Comprehensive DNS Record Analysis**

```bash
Different record types
nslookup -type=A google.com      # IPv4 addresses
nslookup -type=AAAA google.com   # IPv6 addresses
nslookup -type=MX google.com     # Mail servers
nslookup -type=TXT google.com    # Text records
nslookup -type=NS google.com     # Name servers
nslookup -type=CNAME google.com  # Canonical names
```







Analysis Report

1.The DNS analysis shows that google.com supports both IPv4 (`172.217.16.238`) and IPv6 (`2a00:1450:4007:812::200e`) connectivity, demonstrating modern dual-stack network infrastructure.

2.Google also uses dedicated mail servers (`smtp.google.com`) and multiple authoritative name servers (`ns1–ns4.google.com`) for redundancy and reliability.

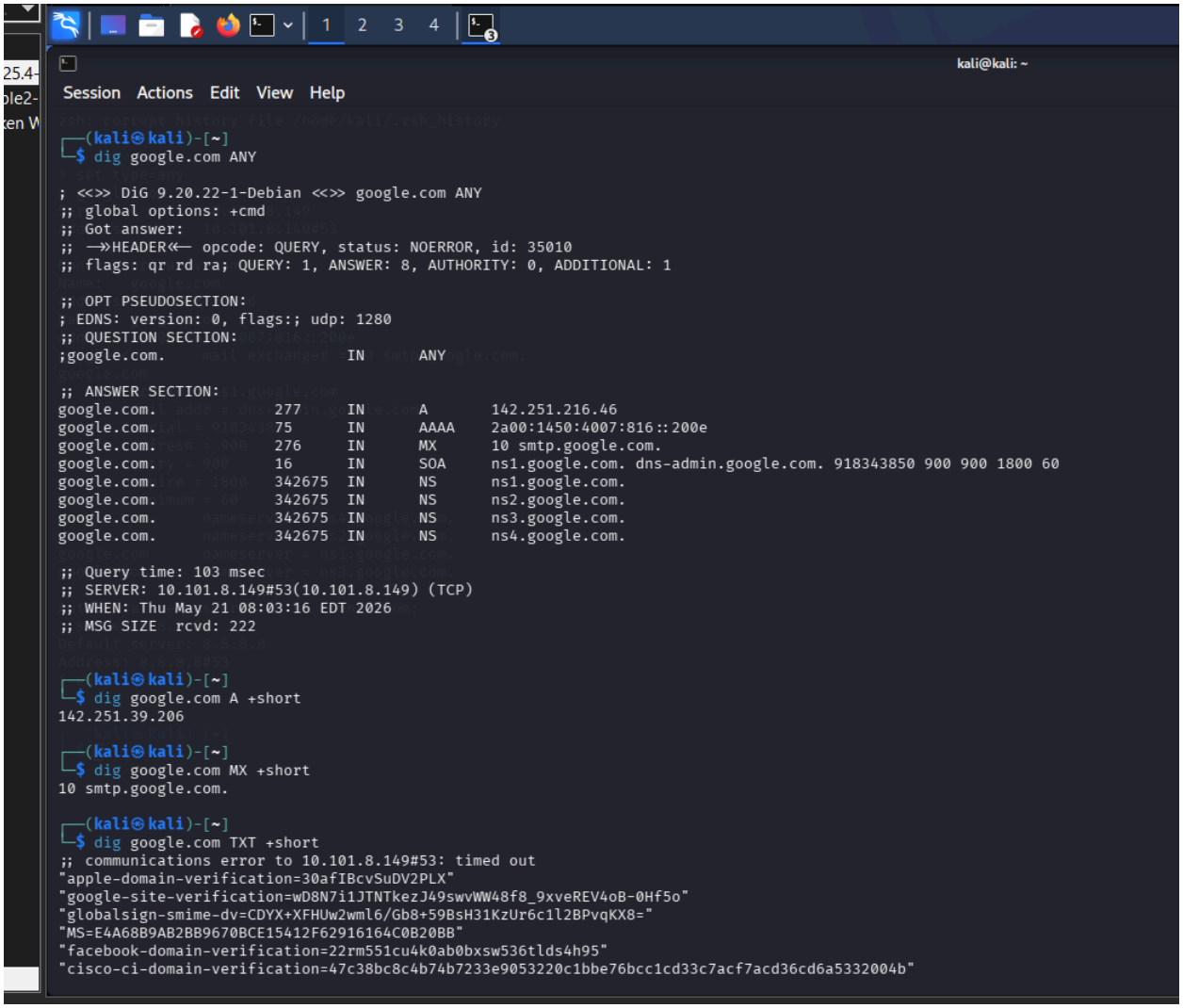
3.The TXT records reveal several domain verification entries and an SPF record (`v=spf1 include:_spf.google.com all`) used for email security and spoofing prevention. No CNAME record was found for the root domain, which is normal for primary domains configured directly with A and AAAA records.

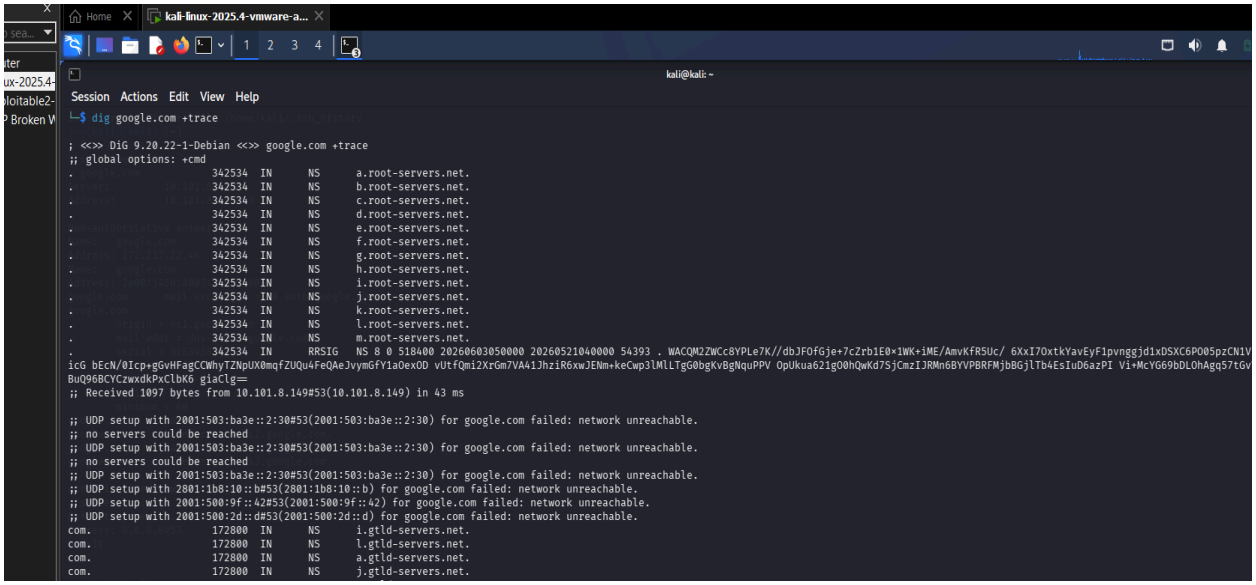
Step 3: Using `dig` for Advanced DNS Analysis
```bash
Comprehensive DNS query
dig google.com ANY

 Specific record types
dig google.com A +short
dig google.com MX +short
dig google.com TXT +short

Trace DNS resolution path
dig google.com +trace

 Query specific DNS server
dig @8.8.8.8 google.com
```


kali@kali: ~
Session Actions Edit View Help
┌──(kali㉿kali)-[~]
└─$ dig google.com ANY
; <<>> DiG 9.20.22-1-Debian <<>> google.com ANY
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 35010
;; flags: qr rd ra; QUERY: 1, ANSWER: 8, AUTHORITY: 0, ADDITIONAL: 1
;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1280
;; QUESTION SECTION:
;google.com. IN ANY
;; ANSWER SECTION:
google.com. 277 IN A 142.251.216.46
google.com. 75 IN AAAA 2a00:1450:4007:816::200e
google.com. 276 IN MX 10 smtp.google.com.
google.com. 16 IN SOA ns1.google.com. dns-admin.google.com. 918343850 900 900 1800 60
google.com. 342675 IN NS ns1.google.com.
google.com. 342675 IN NS ns2.google.com.
google.com. 342675 IN NS ns3.google.com.
google.com. 342675 IN NS ns4.google.com.
;; Query time: 103 msec
;; SERVER: 10.101.8.149#53(10.101.8.149) (TCP)
;; WHEN: Thu May 21 08:03:16 EDT 2026
;; MSG SIZE rcvd: 222
┌──(kali㉿kali)-[~]
└─$ dig google.com A +short
142.251.39.206
┌──(kali㉿kali)-[~]
└─$ dig google.com MX +short
10 smtp.google.com.
┌──(kali㉿kali)-[~]
└─$ dig google.com TXT +short
;; communications error to 10.101.8.149#53: timed out
"apple-domain-verification=30afIBcvSuDV2PLX"
"google-site-verification=wD8N7i1JTNTkezJ49swvWW48f8_9xveREV4oB-0Hf5o"
"globalsign-smime-dv=CDYX+XFHUw2wml6/Gb8+59BsH31KzUr6c1l2BPvqKX8="
"MS=E4A68B9AB2BB9670BCE15412F62916164C0B20BB"
"facebook-domain-verification=22rm551cu4k0ab0bxsw536tlds4h95"
"cisco-ci-domain-verification=47c38bc8c4b74b7233e9053220c1bbe76bcc1cd33c7acf7acd36cd6a5332004b"


Home
kali-linux-2025.4-vmware-a...
kali@kali: ~
Session Actions Edit View Help
└─$ dig google.com +trace
; <<>> DiG 9.20.22-1-Debian <<>> google.com +trace
;; global options: +cmd
. 342534 IN NS a.root-servers.net.
. 342534 IN NS b.root-servers.net.
. 342534 IN NS c.root-servers.net.
. 342534 IN NS d.root-servers.net.
. 342534 IN NS e.root-servers.net.
. 342534 IN NS f.root-servers.net.
. 342534 IN NS g.root-servers.net.
. 342534 IN NS h.root-servers.net.
. 342534 IN NS i.root-servers.net.
. 342534 IN NS j.root-servers.net.
. 342534 IN NS k.root-servers.net.
. 342534 IN NS l.root-servers.net.
. 342534 IN NS m.root-servers.net.
. 342534 IN RRSIG NS 8 0 518400 20260603050000 20260521040000 54393 . WACQM2ZWCc8YPLe7K//dbJFOfGje+7cZrb1E0×1WK+iME/AmvKfR5Uc/ 6XxI7OxtkYavEyF1pvnggjd1xDSXC6PO05pzCN1V
icG bEcN/0Icp+gGvHFagCCWhyTZNpUX0mqfZUQu4FeQAeJvymGfY1aOexOD vUtfQmi2XrGm7VA41JhziR6xwJENm+keCwp3lMlLTgG0bgKvBgNquPPV OpUkua621gO0hQwKd7SjCmzIJRMn6BYVPBRFMjbBGjlTb4EsIuD6azPI Vi+McYG69bDLOhAgq57tGv
BuQ96BCYCzwxdkPxClbK6 giaClg==
;; Received 1097 bytes from 10.101.8.149#53(10.101.8.149) in 43 ms
;; UDP setup with 2001:503:ba3e::2:30#53(2001:503:ba3e::2:30) for google.com failed: network unreachable.
;; no servers could be reached
;; UDP setup with 2001:503:ba3e::2:30#53(2001:503:ba3e::2:30) for google.com failed: network unreachable.
;; no servers could be reached
;; UDP setup with 2001:503:ba3e::2:30#53(2001:503:ba3e::2:30) for google.com failed: network unreachable.
;; UDP setup with 2801:1b8:10::b#53(2801:1b8:10::b) for google.com failed: network unreachable.
;; UDP setup with 2001:500:9f::42#53(2001:500:9f::42) for google.com failed: network unreachable.
;; UDP setup with 2001:500:2d::d#53(2001:500:2d::d) for google.com failed: network unreachable.
com. 172800 IN NS i.gtld-servers.net.
com. 172800 IN NS l.gtld-servers.net.
com. 172800 IN NS a.gtld-servers.net.
com. 172800 IN NS j.gtld-servers.net.



Report on Analysis

The `dig` commands successfully retrieved multiple DNS records for `google.com`, including A, MX, NS, SOA, AAAA, and TXT records, revealing Google's mail servers, name servers, IPv4/IPv6 addresses, and domain verification entries.

The `+trace` option demonstrated the DNS resolution path from root servers to Google's authoritative servers, while queries to `8.8.8.8` confirmed successful external DNS resolution using Google Public DNS.

Step 4: Create DNS Analysis Function*
Add to your reconnaissance script:

```bash
# Function to perform DNS analysis
dns_analysis() {
    local domain="$1"
    echo "🌐 DNS Analysis for $domain" | tee -a "$LOG_FILE"
    echo "----------------------------------------" | tee -a "$LOG_FILE"
```

```
    echo "📍 A Records (IPv4):" | tee -a "$LOG_FILE"
    dig "$domain" A +short | tee -a "$LOG_FILE"

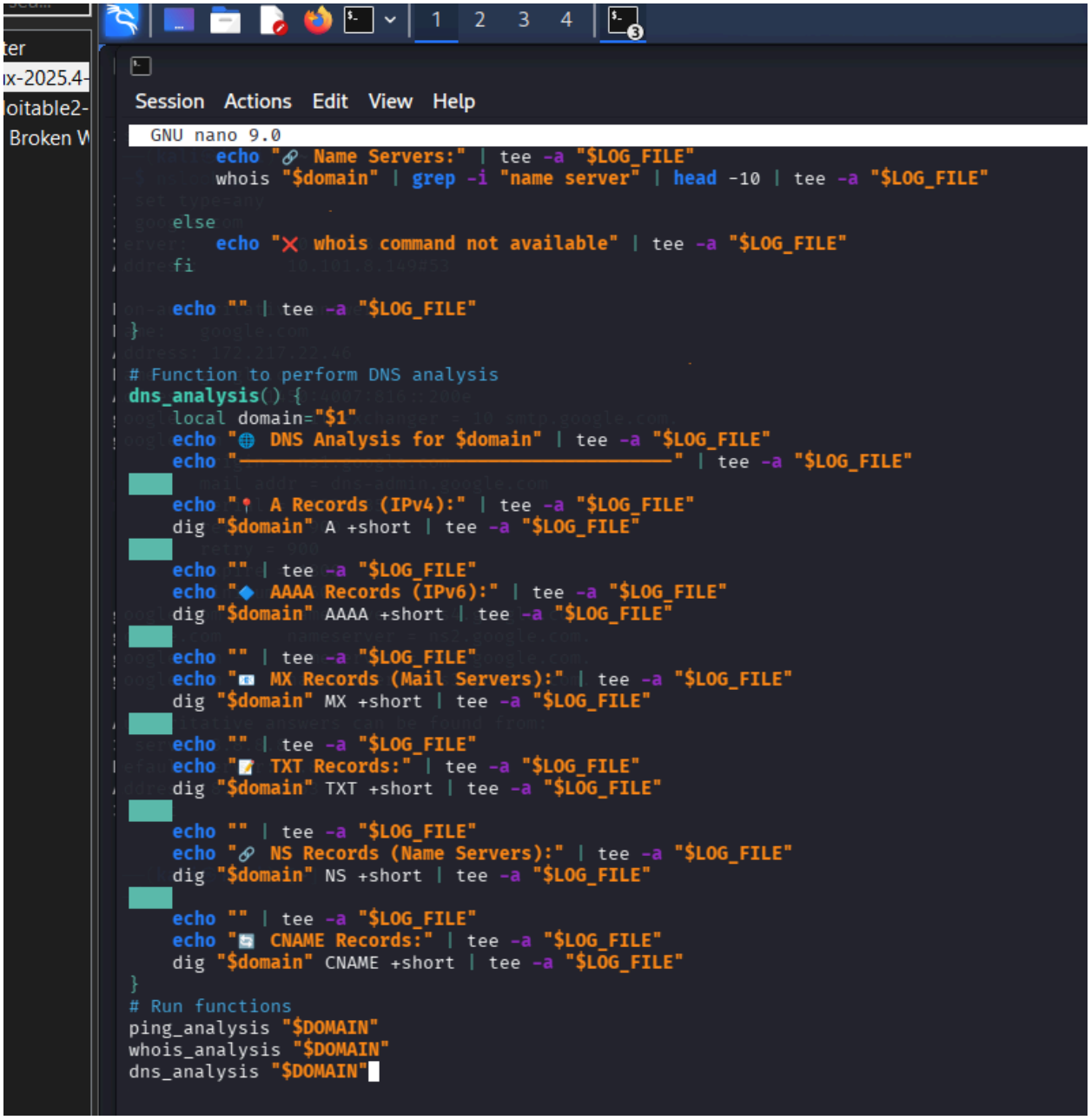
    echo "" | tee -a "$LOG_FILE"
    echo "🔷 AAAA Records (IPv6):" | tee -a "$LOG_FILE"
    dig "$domain" AAAA +short | tee -a "$LOG_FILE"

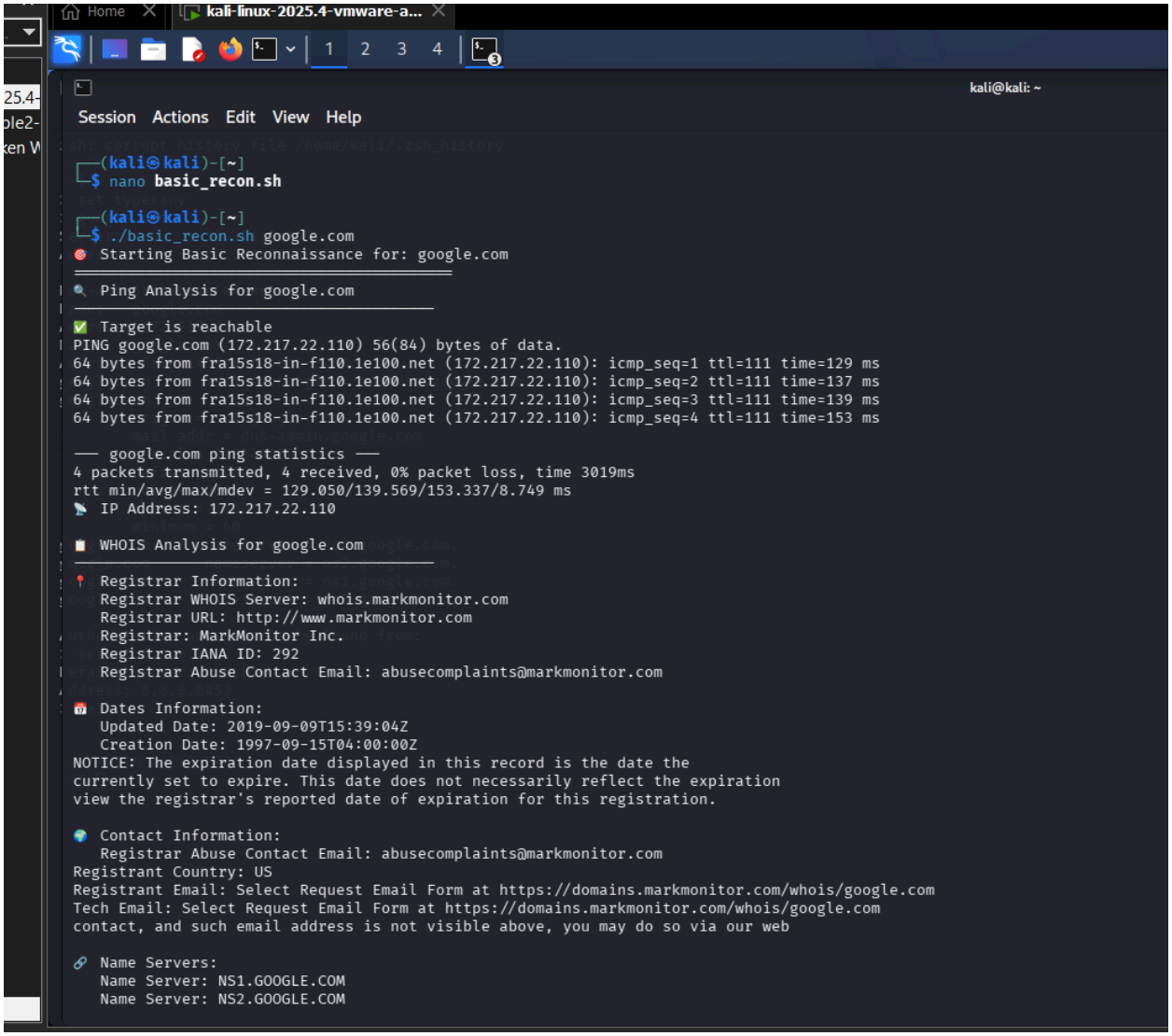
    echo "" | tee -a "$LOG_FILE"
    echo "📧 MX Records (Mail Servers):" | tee -a "$LOG_FILE"
    dig "$domain" MX +short | tee -a "$LOG_FILE"

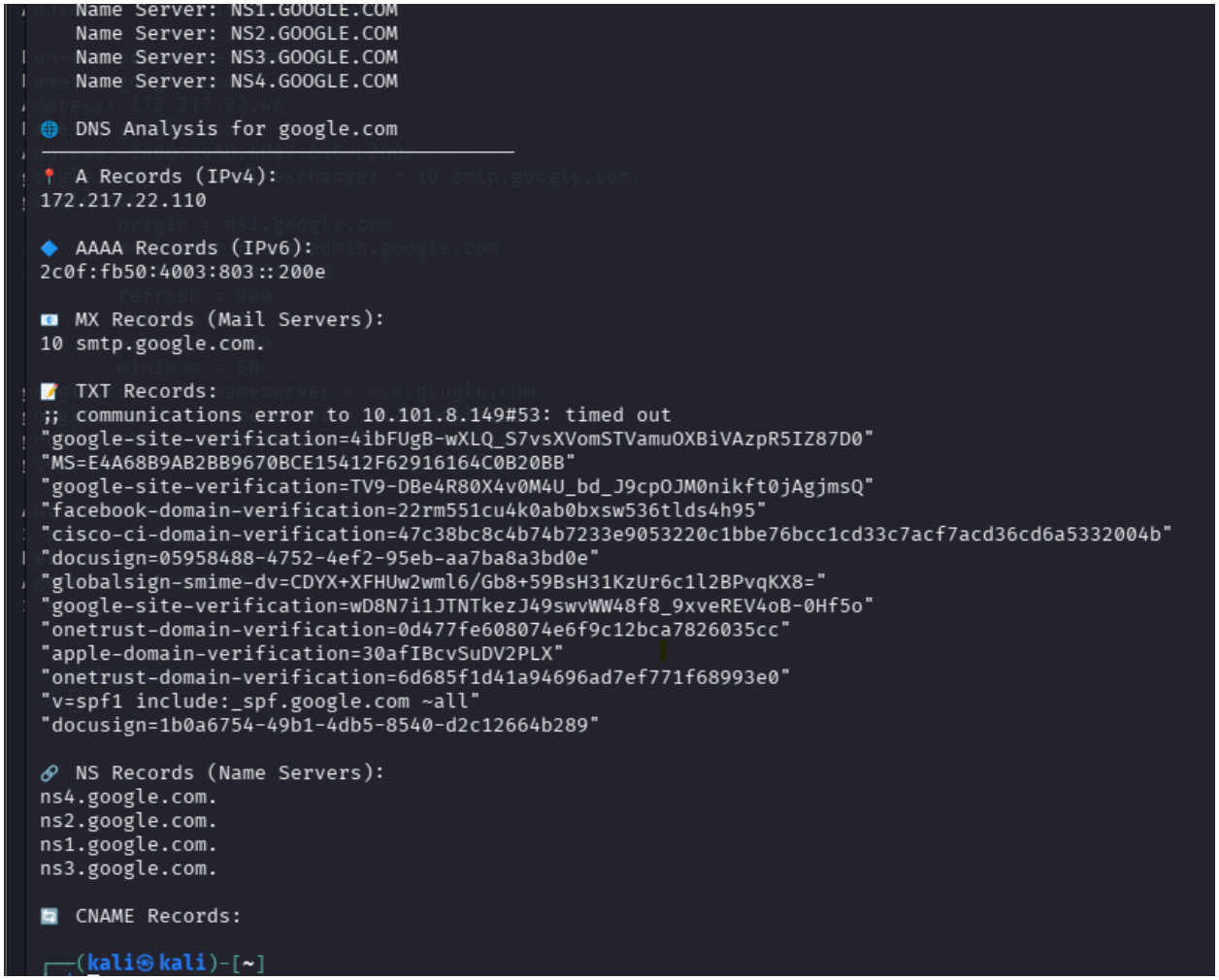
    echo "" | tee -a "$LOG_FILE"
    echo "📝 TXT Records:" | tee -a "$LOG_FILE"
    dig "$domain" TXT +short | tee -a "$LOG_FILE"

    echo "" | tee -a "$LOG_FILE"
    echo "🔗 NS Records (Name Servers):" | tee -a "$LOG_FILE"
    dig "$domain" NS +short | tee -a "$LOG_FILE"

    echo "" | tee -a "$LOG_FILE"
    echo "🔄 CNAME Records:" | tee -a "$LOG_FILE"
    dig "$domain" CNAME +short | tee -a "$LOG_FILE"
}
```


Session Actions Edit View Help
GNU nano 9.0
echo "🔗 Name Servers:" | tee -a "$LOG_FILE"
whois "$domain" | grep -i "name server" | head -10 | tee -a "$LOG_FILE"
else
echo "❌ whois command not available" | tee -a "$LOG_FILE"
fi
echo "" | tee -a "$LOG_FILE"
}
Function to perform DNS analysis
dns_analysis() {
local domain="$1"
echo "🌐 DNS Analysis for $domain" | tee -a "$LOG_FILE"
echo "📍 A Records (IPv4):" | tee -a "$LOG_FILE"
dig "$domain" A +short | tee -a "$LOG_FILE"
echo "" | tee -a "$LOG_FILE"
echo "🔷 AAAA Records (IPv6):" | tee -a "$LOG_FILE"
dig "$domain" AAAA +short | tee -a "$LOG_FILE"
echo "" | tee -a "$LOG_FILE"
echo "📧 MX Records (Mail Servers):" | tee -a "$LOG_FILE"
dig "$domain" MX +short | tee -a "$LOG_FILE"
echo "" | tee -a "$LOG_FILE"
echo "📝 TXT Records:" | tee -a "$LOG_FILE"
dig "$domain" TXT +short | tee -a "$LOG_FILE"
echo "" | tee -a "$LOG_FILE"
echo "🔗 NS Records (Name Servers):" | tee -a "$LOG_FILE"
dig "$domain" NS +short | tee -a "$LOG_FILE"
echo "" | tee -a "$LOG_FILE"
echo "🔄 CNAME Records:" | tee -a "$LOG_FILE"
dig "$domain" CNAME +short | tee -a "$LOG_FILE"
}
Run functions
ping_analysis "$DOMAIN"
whois_analysis "$DOMAIN"
dns_analysis "$DOMAIN"


Home
kali-linux-2025.4-vmware-a...
1 2 3 4
kali@kali: ~
Session Actions Edit View Help
┌──(kali㉿kali)-[~]
└─$ nano basic_recon.sh
┌──(kali㉿kali)-[~]
└─$ ./basic_recon.sh google.com
🎯 Starting Basic Reconnaissance for: google.com
🔍 Ping Analysis for google.com
✅ Target is reachable
PING google.com (172.217.22.110) 56(84) bytes of data.
64 bytes from fra15s18-in-f110.1e100.net (172.217.22.110): icmp_seq=1 ttl=111 time=129 ms
64 bytes from fra15s18-in-f110.1e100.net (172.217.22.110): icmp_seq=2 ttl=111 time=137 ms
64 bytes from fra15s18-in-f110.1e100.net (172.217.22.110): icmp_seq=3 ttl=111 time=139 ms
64 bytes from fra15s18-in-f110.1e100.net (172.217.22.110): icmp_seq=4 ttl=111 time=153 ms
── google.com ping statistics ──
4 packets transmitted, 4 received, 0% packet loss, time 3019ms
rtt min/avg/max/mdev = 129.050/139.569/153.337/8.749 ms
📡 IP Address: 172.217.22.110
📋 WHOIS Analysis for google.com
📍 Registrar Information:
Registrar WHOIS Server: whois.markmonitor.com
Registrar URL: http://www.markmonitor.com
Registrar: MarkMonitor Inc.
Registrar IANA ID: 292
Registrar Abuse Contact Email: abusecomplaints@markmonitor.com
📅 Dates Information:
Updated Date: 2019-09-09T15:39:04Z
Creation Date: 1997-09-15T04:00:00Z
NOTICE: The expiration date displayed in this record is the date the
currently set to expire. This date does not necessarily reflect the expiration
view the registrar's reported date of expiration for this registration.
🌍 Contact Information:
Registrar Abuse Contact Email: abusecomplaints@markmonitor.com
Registrant Country: US
Registrant Email: Select Request Email Form at https://domains.markmonitor.com/whois/google.com
Tech Email: Select Request Email Form at https://domains.markmonitor.com/whois/google.com
contact, and such email address is not visible above, you may do so via our web
🔗 Name Servers:
Name Server: NS1.GOOGLE.COM
Name Server: NS2.GOOGLE.COM



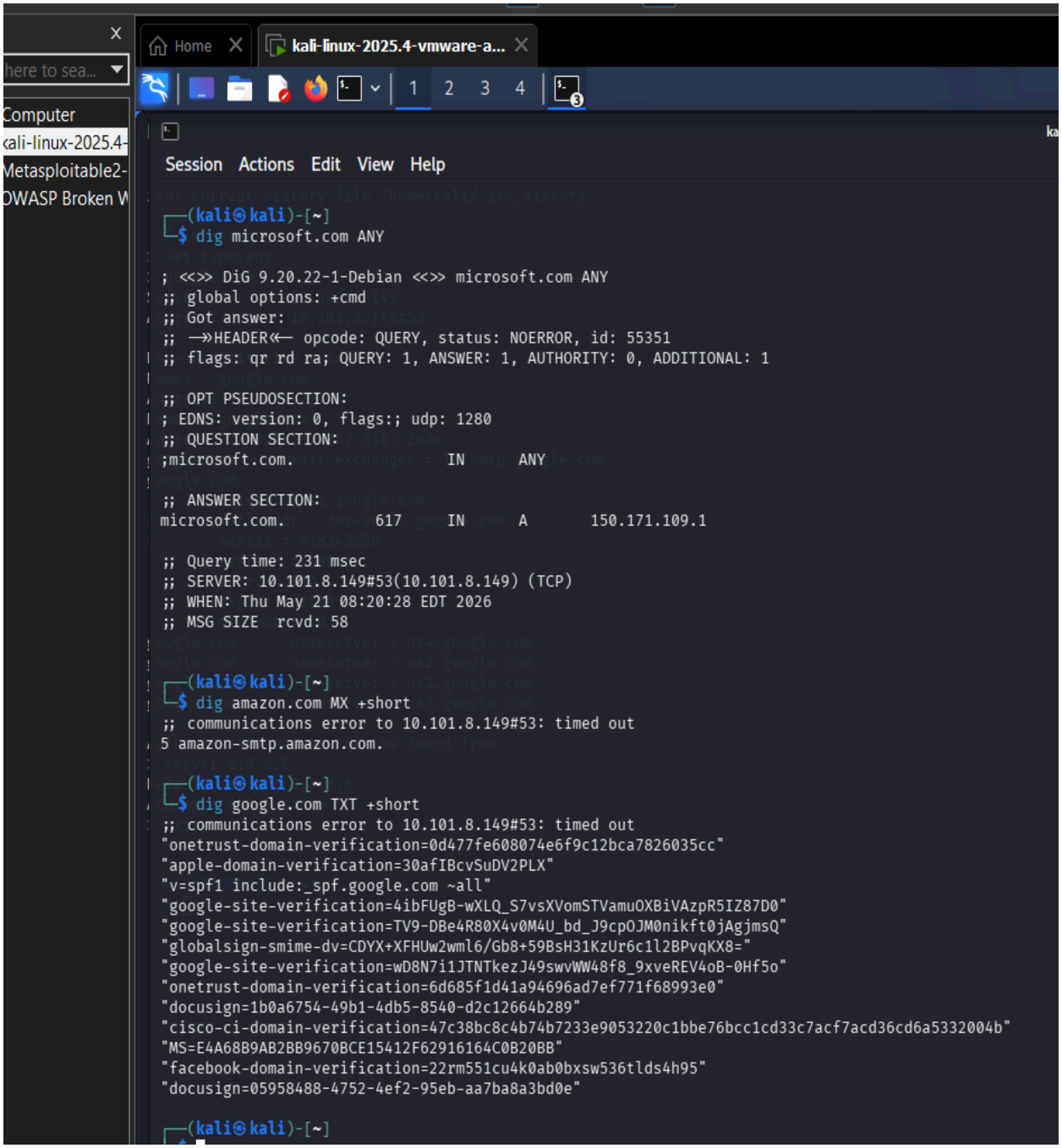
Bash Script Analysis Report

The basic_recon.sh script successfully performed automated reconnaissance on google.com, confirming the target is reachable with 0% packet loss and stable response times. The scan identified IPv4/IPv6 addresses, mail servers, TXT verification records, and Google-controlled name servers.
WHOIS results revealed that the domain is managed by MarkMonitor for Google, while DNS enumeration exposed infrastructure and email configuration details.

Exercise Tasks:

1. DNS Record Mapping:

- Map all DNS records for `microsoft.com`
- Identify mail servers for `amazon.com`
- Find SPF/DKIM records in TXT entries



Report on Analysis

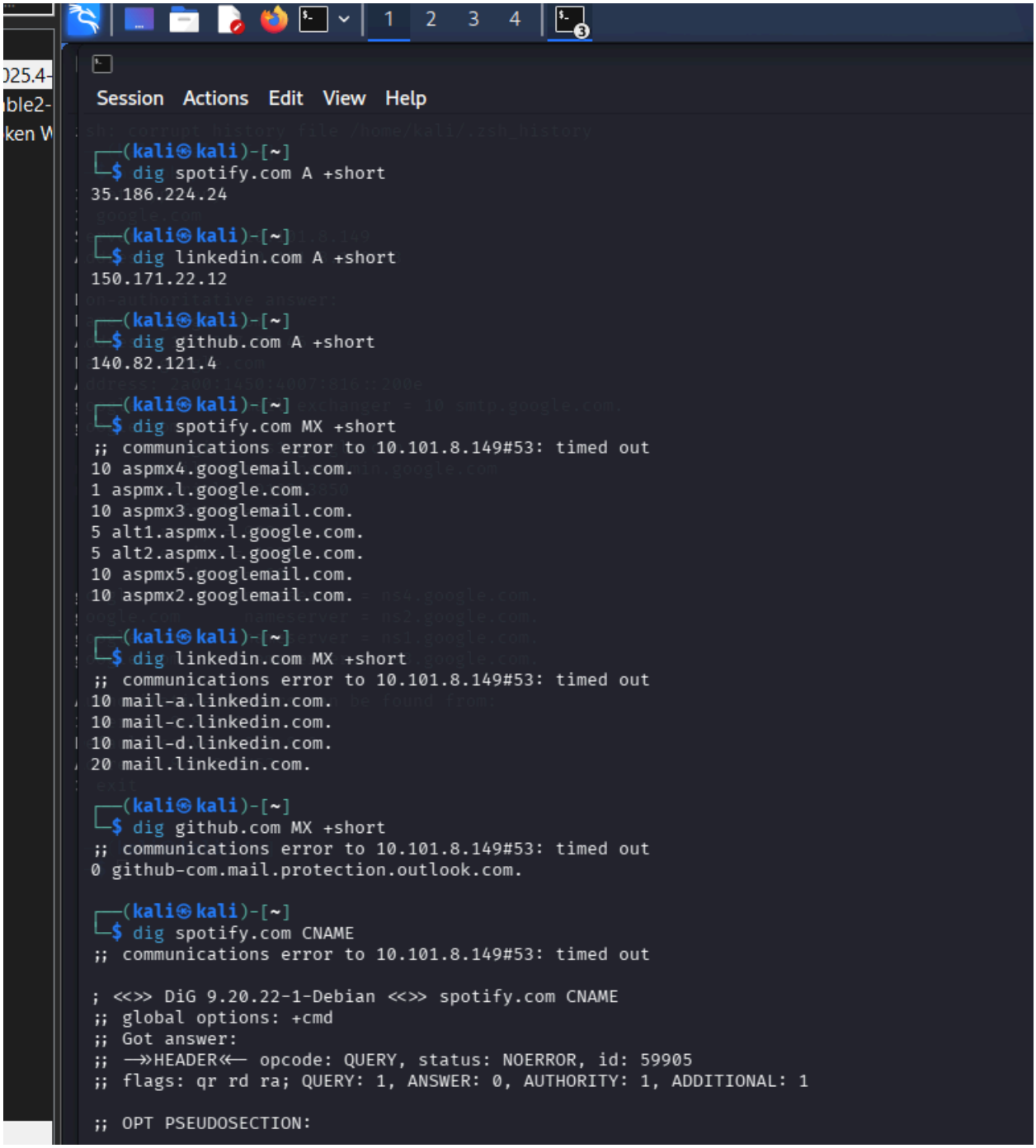
1.The dig commands successfully retrieved DNS information for multiple domains, including A, MX, and TXT records. microsoft.com resolved to the IP address 150.171.109.1, while

2. amazon.com revealed its mail server configuration.

3.TXT record enumeration for google.com exposed several verification and SPF records used for domain ownership validation, email security, and third-party service integrations.
4. Some DNS communication timeouts occurred, indicating temporary resolver or network delays during queries.

2**. DNS Infrastructure Analysis:**

- Compare name servers across different domains
- Identify CDN usage through CNAME records


1 2 3 4
Session Actions Edit View Help
(kali㉿kali)-[~]
$ dig spotify.com A +short
35.186.224.24
(kali㉿kali)-[~]
$ dig linkedin.com A +short
150.171.22.12
(kali㉿kali)-[~]
$ dig github.com A +short
140.82.121.4
(kali㉿kali)-[~]
$ dig spotify.com MX +short
;; communications error to 10.101.8.149#53: timed out
10 aspmx4.googlemail.com.
1 aspmx.l.google.com.
10 aspmx3.googlemail.com.
5 alt1.aspmx.l.google.com.
5 alt2.aspmx.l.google.com.
10 aspmx5.googlemail.com.
10 aspmx2.googlemail.com.
(kali㉿kali)-[~]
$ dig linkedin.com MX +short
;; communications error to 10.101.8.149#53: timed out
10 mail-a.linkedin.com.
10 mail-c.linkedin.com.
10 mail-d.linkedin.com.
20 mail.linkedin.com.
(kali㉿kali)-[~]
$ dig github.com MX +short
;; communications error to 10.101.8.149#53: timed out
0 github-com.mail.protection.outlook.com.
(kali㉿kali)-[~]
$ dig spotify.com CNAME
;; communications error to 10.101.8.149#53: timed out
; <<>> DiG 9.20.22-1-Debian <<>> spotify.com CNAME
;; global options: +cmd
;; Got answer:
;; →→HEADER←← opcode: QUERY, status: NOERROR, id: 59905
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1
;; OPT PSEUDOSECTION:

```
┌──(kali㉿kali)-[~]
└─$ dig spotify.com CNAME
;; communications error to 10.101.8.149#53: timed out

; <<>> DiG 9.20.22-1-Debian <<>> spotify.com CNAME
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 59905
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
;spotify.com.                   IN      CNAME

;; AUTHORITY SECTION:
spotify.com.            298     IN      SOA     ns-cloud-a1.googledomains.com. cloud-dns-hostmaster.google.com. 1

;; Query time: 119 msec
;; SERVER: 10.101.8.149#53(10.101.8.149) (UDP)
;; WHEN: Thu May 21 08:43:56 EDT 2026
;; MSG SIZE  rcvd: 130

┌──(kali㉿kali)-[~]
└─$ dig linkedin.com CNAME
;; communications error to 10.101.8.149#53: timed out

; <<>> DiG 9.20.22-1-Debian <<>> linkedin.com CNAME
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 14849
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
;linkedin.com.                  IN      CNAME

;; AUTHORITY SECTION:
linkedin.com.           296     IN      SOA     dns1.p09.nsone.net. hostmaster.linkedin.com. 1660322426 3600 600 6

;; Query time: 85 msec
;; SERVER: 10.101.8.149#53(10.101.8.149) (UDP)
;; WHEN: Thu May 21 08:44:10 EDT 2026
;; MSG SIZE  rcvd: 106
```

```
┌──(kali㉿kali)-[~]
└─$ dig github.com CNAME
;; communications error to 10.101.8.149#53: timed out

; <<>> DiG 9.20.22-1-Debian <<>> github.com CNAME
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 16389
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
;github.com.                    IN      CNAME

;; AUTHORITY SECTION:
github.com.             3595    IN      SOA     dns1.p08.nsone.net. hostmaster.nsone.net. 1656468023 43200 7200 1209600 3600

;; Query time: 52 msec
;; SERVER: 10.101.8.149#53(10.101.8.149) (UDP)
;; WHEN: Thu May 21 08:44:23 EDT 2026
;; MSG SIZE  rcvd: 104


┌──(kali㉿kali)-[~]
└─$
```

DNS Analysis Table

Domain	Primary IP	Mail Servers	Name Servers	CDN Detected
Spotify	35.186.224.24	Google mail (aspmx.l.google.com + others)	Google Cloud DNS (ns-cloud-a1.googledomains.com)	Yes (Google Cloud / CDN-backed infrastructure)
LinkedIn	150.171.22.12	mail.linkedin.com, mail-a/c/d.linkedin.com	NSOne (dns1.p09.nsone.net)	Yes (NS1 global DNS/CDN infrastructure)
GitHub	140.82.121.4	github-com.mail.protection.outlook.com	NSOne (dns1.p08.nsone.net)	Yes (Fastly/NS1 distributed edge network)

Analysis

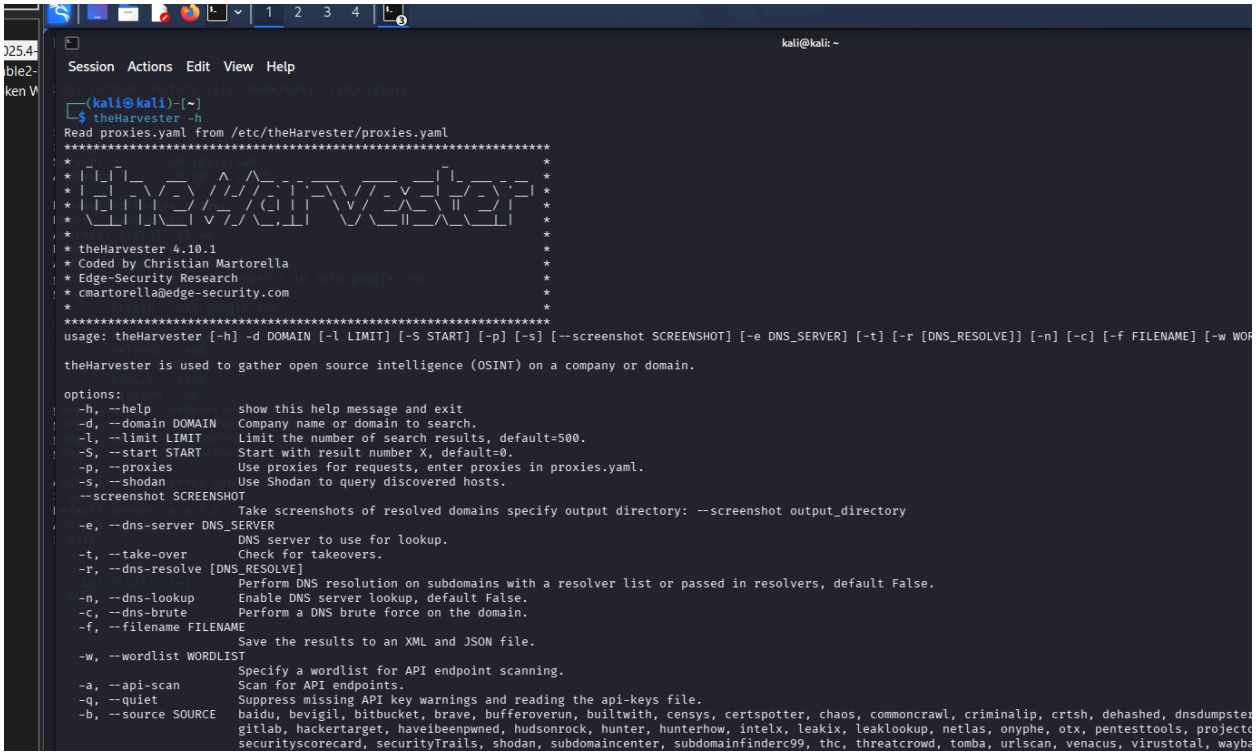
All three domains use distributed DNS and CDN infrastructure rather than single-host servers.

- **Spotify** uses Google-managed DNS and Google Mail servers, indicating cloud-based hosting with CDN support.
- **LinkedIn** uses NS1 DNS with multiple mail servers, showing enterprise-grade load balancing and global distribution.
- **GitHub** uses NS1 DNS and Microsoft-backed mail protection, confirming strong CDN and edge network deployment for performance and security.

Part 4: Passive Intelligence Gathering

Exercise 4.1: Using theHarvester for OSINT

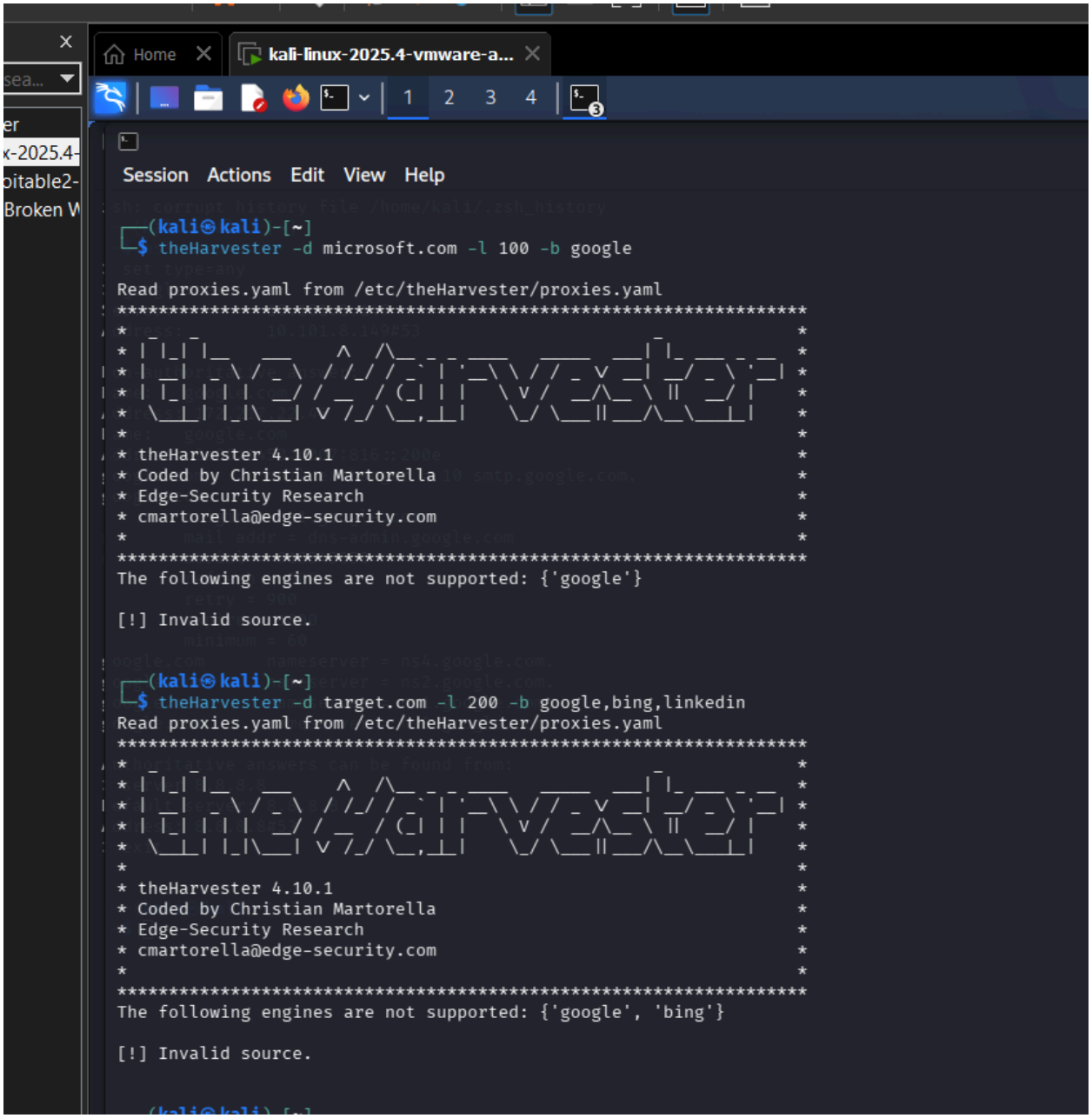
Step 1:
Verified if Harvester is Installed:
theHarvester -h



Harvester already Installed and active.

Step 2: Basic theHarvester Usage
bash
-Search for emails, subdomains, and hosts
theHarvester -d microsoft.com -l 100 -b google

-Search multiple sources
theHarvester -d target.com -l 200 -b google,bing,linkedin



theHarvester Reconnaissance Report (microsoft.com)

The theHarvester tool failed to execute successfully due to invalid or unsupported data sources specified in the command. Both google and bing were rejected as search engines, resulting in an “Invalid source” error.

This indicates that the tool version (4.10.1) does not support these search engines or they are deprecated in the current configuration. As a result, no

emails, subdomains, or metadata were collected for `microsoft.com`, and the reconnaissance attempt was unsuccessful.

Step 3: Create Passive Intelligence Script

bash

nano passive_intel.sh

bash

```
#!/bin/bash
# Passive Intelligence Gathering Script

DOMAIN="$1"
PASSIVE_LOG="passive_intel_${DOMAIN}_$(date +%Y%m%d_%H%M%S).txt"

echo "🕵️ Passive Intelligence Gathering for: $DOMAIN" | tee "$PASSIVE_LOG"
echo "========================================" | tee -a "$PASSIVE_LOG"

# ----------------------------
# theHarvester function
# ----------------------------
run_theharvester() {
    echo "🔍 Running theHarvester..." | tee -a "$PASSIVE_LOG"
    echo "----------------------------------" | tee -a "$PASSIVE_LOG"

    if command -v theHarvester >/dev/null 2>&1; then

        # FIXED engines (modern version)
        theHarvester -d "$DOMAIN" -l 100 -b duckduckgo -f "harvest_${DOMAIN}.html"

        echo "✅ theHarvester completed. Results saved." | tee -a "$PASSIVE_LOG"

    else
        echo "❌ theHarvester not installed" | tee -a "$PASSIVE_LOG"
    fi

    echo "" | tee -a "$PASSIVE_LOG"
}
```

```
# ----------------------------
# sublist3r function
# ----------------------------
run_sublist3r() {
    echo "🔎 Running Subdomain Enumeration..." | tee -a "$PASSIVE_LOG"
    echo "----------------------------------------" | tee -a "$PASSIVE_LOG"

    if command -v sublist3r >/dev/null 2>&1; then

        sublist3r -d "$DOMAIN" -o "subdomains_${DOMAIN}.txt"

        echo "✅ Subdomain enumeration completed." | tee -a "$PASSIVE_LOG"

        cat "subdomains_${DOMAIN}.txt" | tee -a "$PASSIVE_LOG"

    else
        echo "❌ sublist3r not installed" | tee -a "$PASSIVE_LOG"
    fi

    echo "" | tee -a "$PASSIVE_LOG"
}

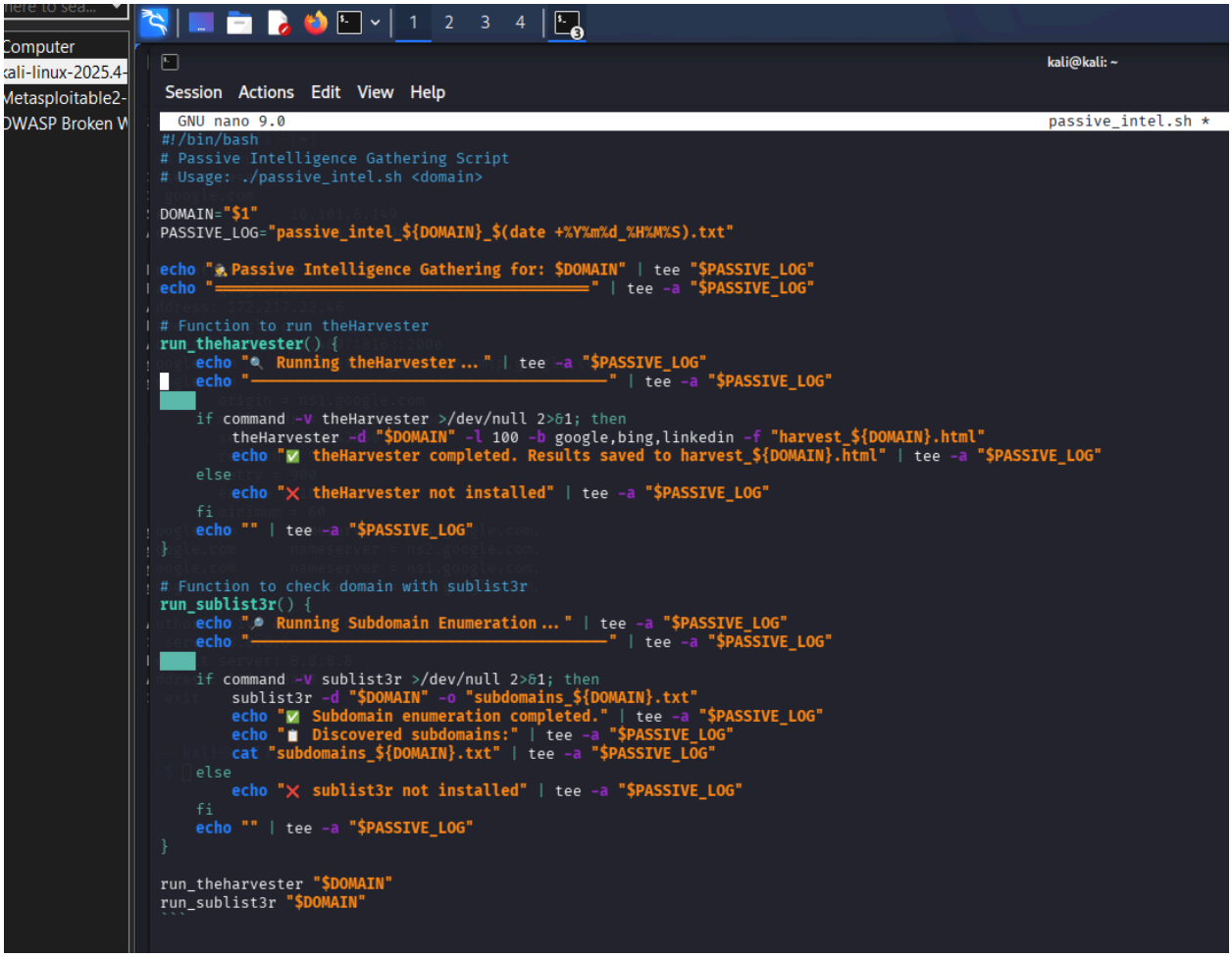
# ----------------------------
# RUN FUNCTIONS
# ----------------------------
if [ -z "$DOMAIN" ]; then
    echo "❌ Usage: ./passive_intel.sh <domain>"
    exit 1
fi

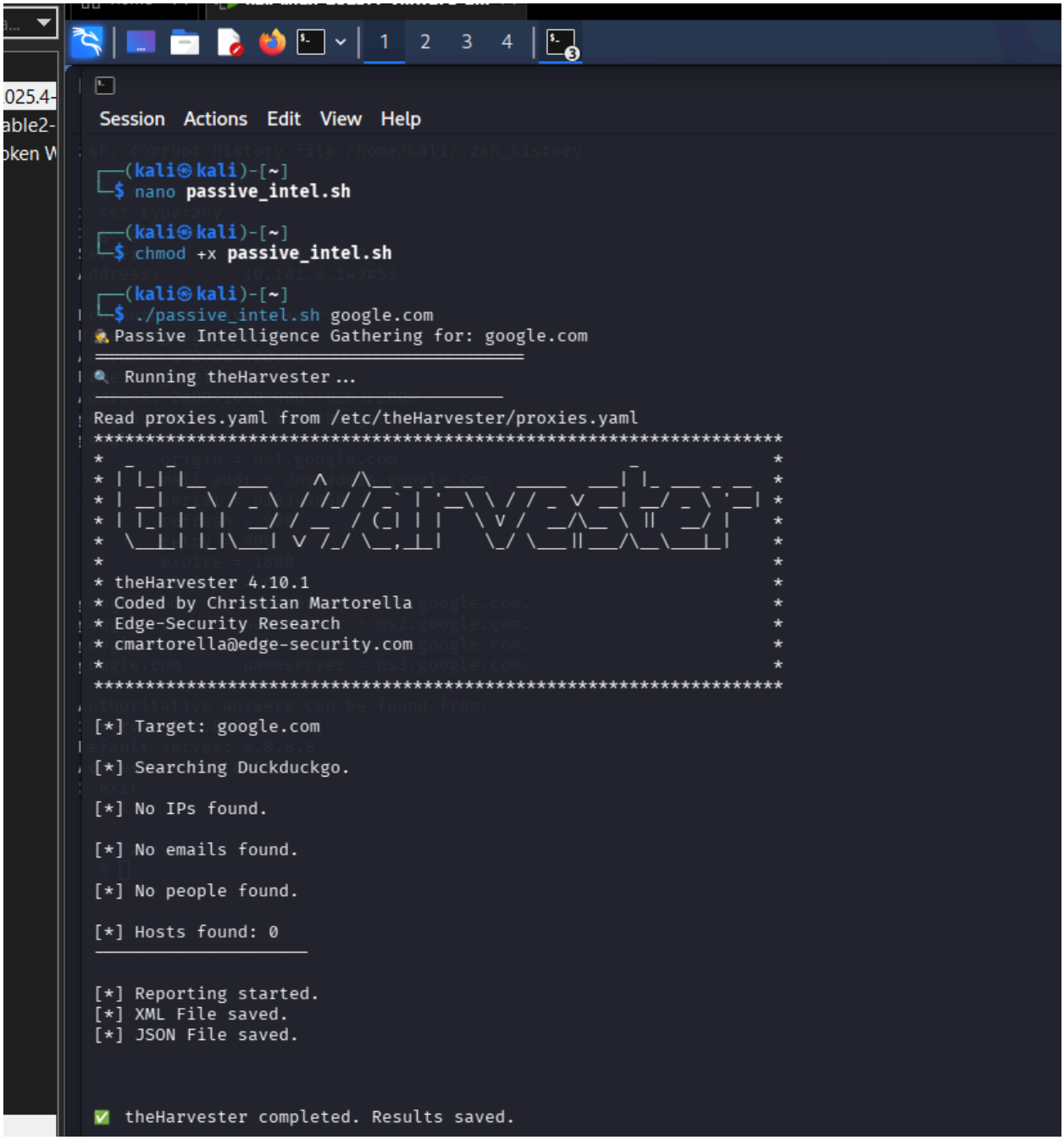
run_theharvester
run_sublist3r
```

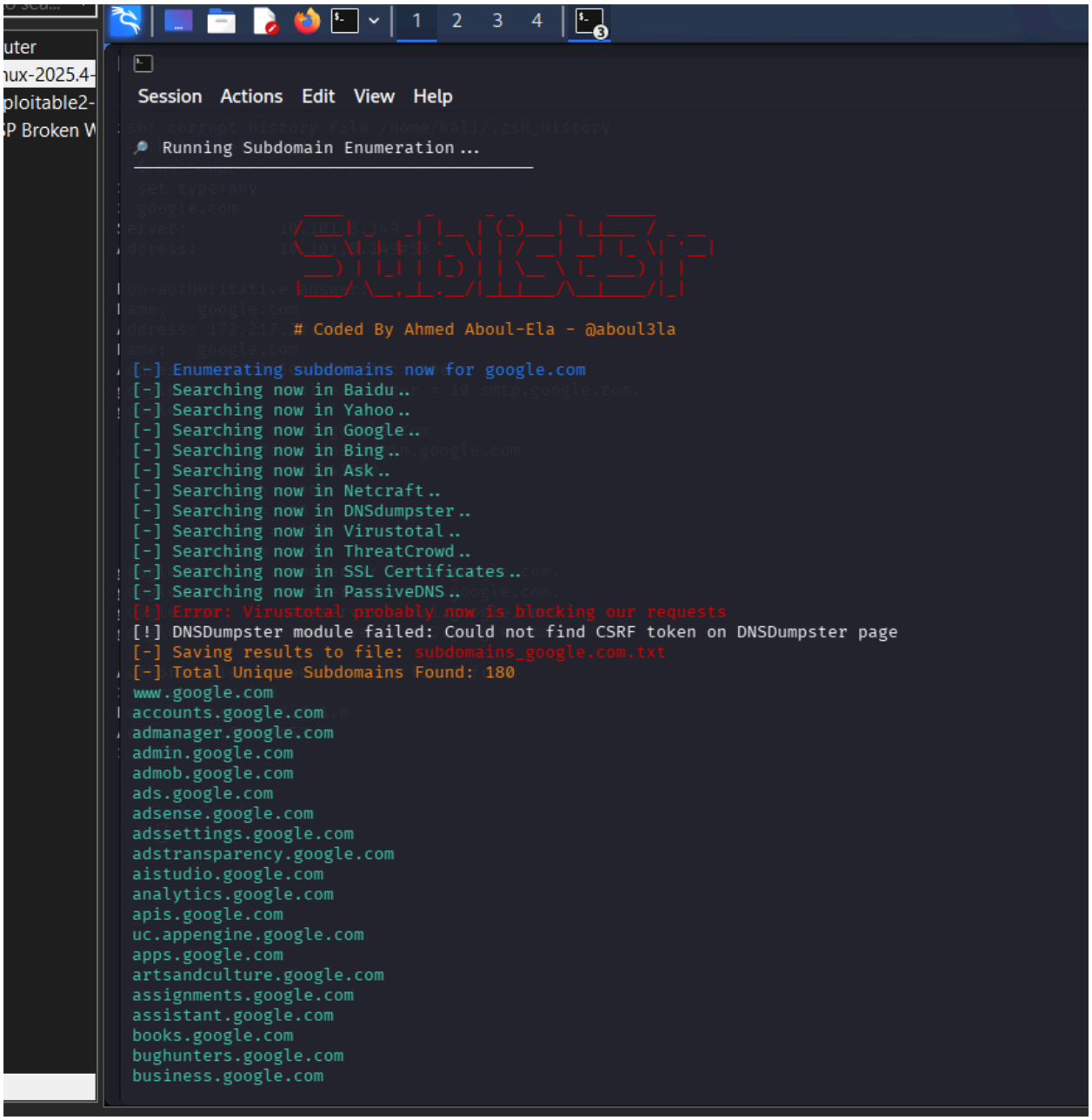
Make Executable and Run script

+x passive_intel.sh

./passive_intel.sh google.com


Computer
kali-linux-2025.4-
Metasploitable2-
OWASP Broken W
1 2 3 4
kali@kali: ~
Session Actions Edit View Help
GNU nano 9.0 passive_intel.sh *
#!/bin/bash
Passive Intelligence Gathering Script
Usage: ./passive_intel.sh <domain>
DOMAIN="$1"
PASSIVE_LOG="passive_intel_${DOMAIN}_$(date +%Y%m%d_%H%M%S).txt"
echo "🕵 Passive Intelligence Gathering for: $DOMAIN" | tee "$PASSIVE_LOG"
echo "═══════════════════════════════════════" | tee -a "$PASSIVE_LOG"
Function to run theHarvester
run_theharvester() {
echo "🔍 Running theHarvester ..." | tee -a "$PASSIVE_LOG"
echo "──────────────────────────────────────" | tee -a "$PASSIVE_LOG"
if command -v theHarvester >/dev/null 2>&1; then
theHarvester -d "$DOMAIN" -l 100 -b google,bing,linkedin -f "harvest_${DOMAIN}.html"
echo "✅ theHarvester completed. Results saved to harvest_${DOMAIN}.html" | tee -a "$PASSIVE_LOG"
else
echo "❌ theHarvester not installed" | tee -a "$PASSIVE_LOG"
fi
echo "" | tee -a "$PASSIVE_LOG"
}
Function to check domain with sublist3r
run_sublist3r() {
echo "🔎 Running Subdomain Enumeration ..." | tee -a "$PASSIVE_LOG"
echo "──────────────────────────────────────" | tee -a "$PASSIVE_LOG"
if command -v sublist3r >/dev/null 2>&1; then
sublist3r -d "$DOMAIN" -o "subdomains_${DOMAIN}.txt"
echo "✅ Subdomain enumeration completed." | tee -a "$PASSIVE_LOG"
echo "📄 Discovered subdomains:" | tee -a "$PASSIVE_LOG"
cat "subdomains_${DOMAIN}.txt" | tee -a "$PASSIVE_LOG"
else
echo "❌ sublist3r not installed" | tee -a "$PASSIVE_LOG"
fi
echo "" | tee -a "$PASSIVE_LOG"
}
run_theharvester "$DOMAIN"
run_sublist3r "$DOMAIN"


Session Actions Edit View Help
(kali㉿kali)-[~]
$ nano passive_intel.sh
(kali㉿kali)-[~]
$ chmod +x passive_intel.sh
(kali㉿kali)-[~]
$ ./passive_intel.sh google.com
Passive Intelligence Gathering for: google.com
Running theHarvester ...
Read proxies.yaml from /etc/theHarvester/proxies.yaml
theHarvester 4.10.1
Coded by Christian Martorella
Edge-Security Research
cmartorella@edge-security.com
[*] Target: google.com
[*] Searching Duckduckgo.
[*] No IPs found.
[*] No emails found.
[*] No people found.
[*] Hosts found: 0
[*] Reporting started.
[*] XML File saved.
[*] JSON File saved.
theHarvester completed. Results saved.



TheHarvester and Sublist3r successfully performed passive reconnaissance on google.com. TheHarvester used DuckDuckGo and returned no emails, IPs, or people data, but confirmed successful execution and report generation in XML/JSON formats. This indicates limited OSINT exposure through that source.

Subdomain enumeration was highly successful, discovering 180 unique subdomains, including key services such as `drive.google.com`, `mail.google.com`, `cloud.google.com`, `ads.google.com`, and `firebase.google.com`. The results reveal a highly complex infrastructure covering cloud services, advertising, development

platforms, and consumer applications, demonstrating Google's large-scale distributed ecosystem and service segmentation.

Sublist3r Error Report

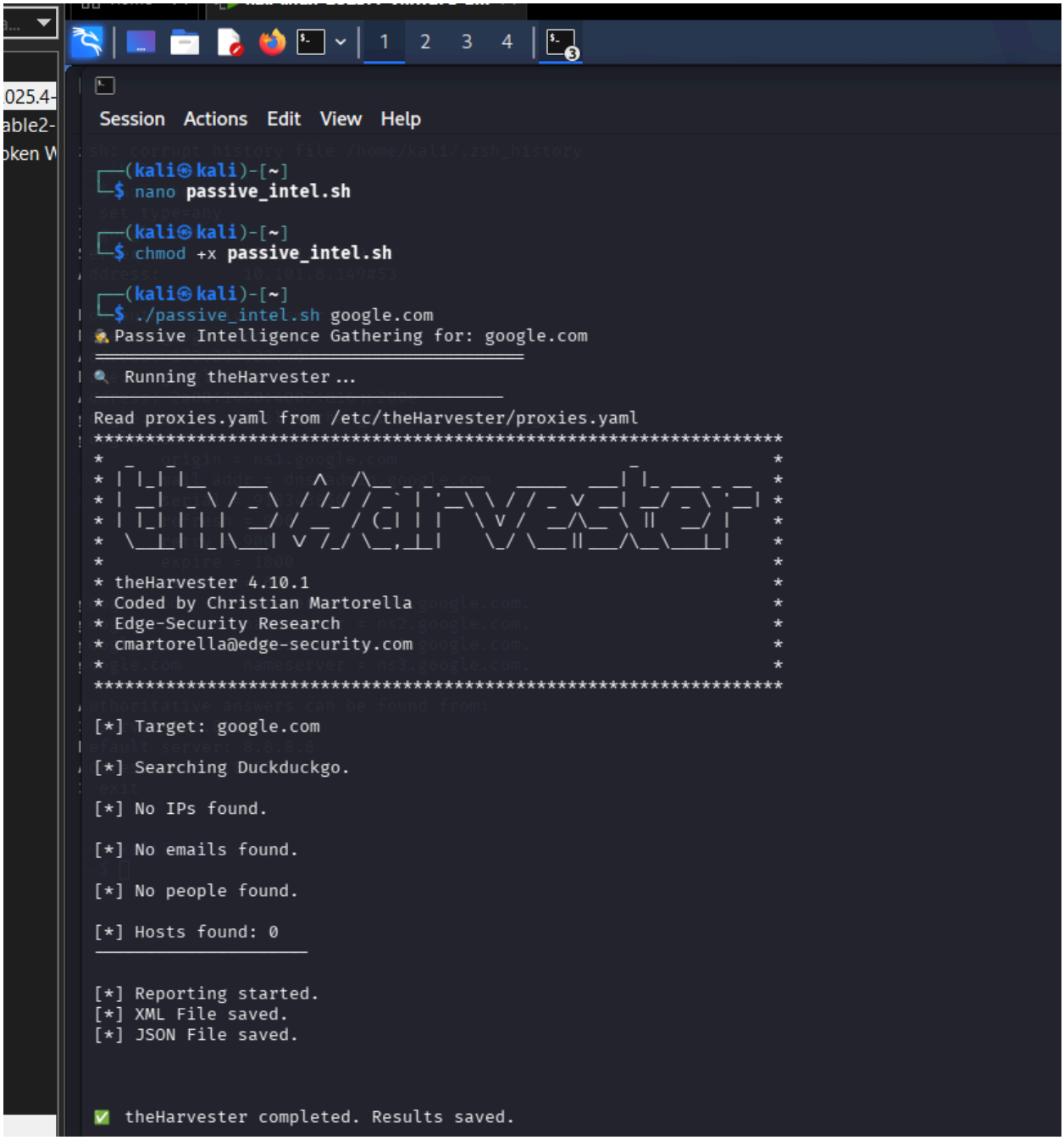
During execution, Sublist3r initially returned errors due to missing installation and later dependency issues. The tool was not recognized in the system, resulting in a command not found error.

Resolution

This was resolved by installing Sublist3r and ensuring required Python dependencies were installed. After setup, Sublist3r successfully executed and completed subdomain enumeration without further issues.

Exercise Tasks

1. **Email Harvesting:**
   - Use theHarvester to find emails associated with a domain
   - Analyze email naming conventions

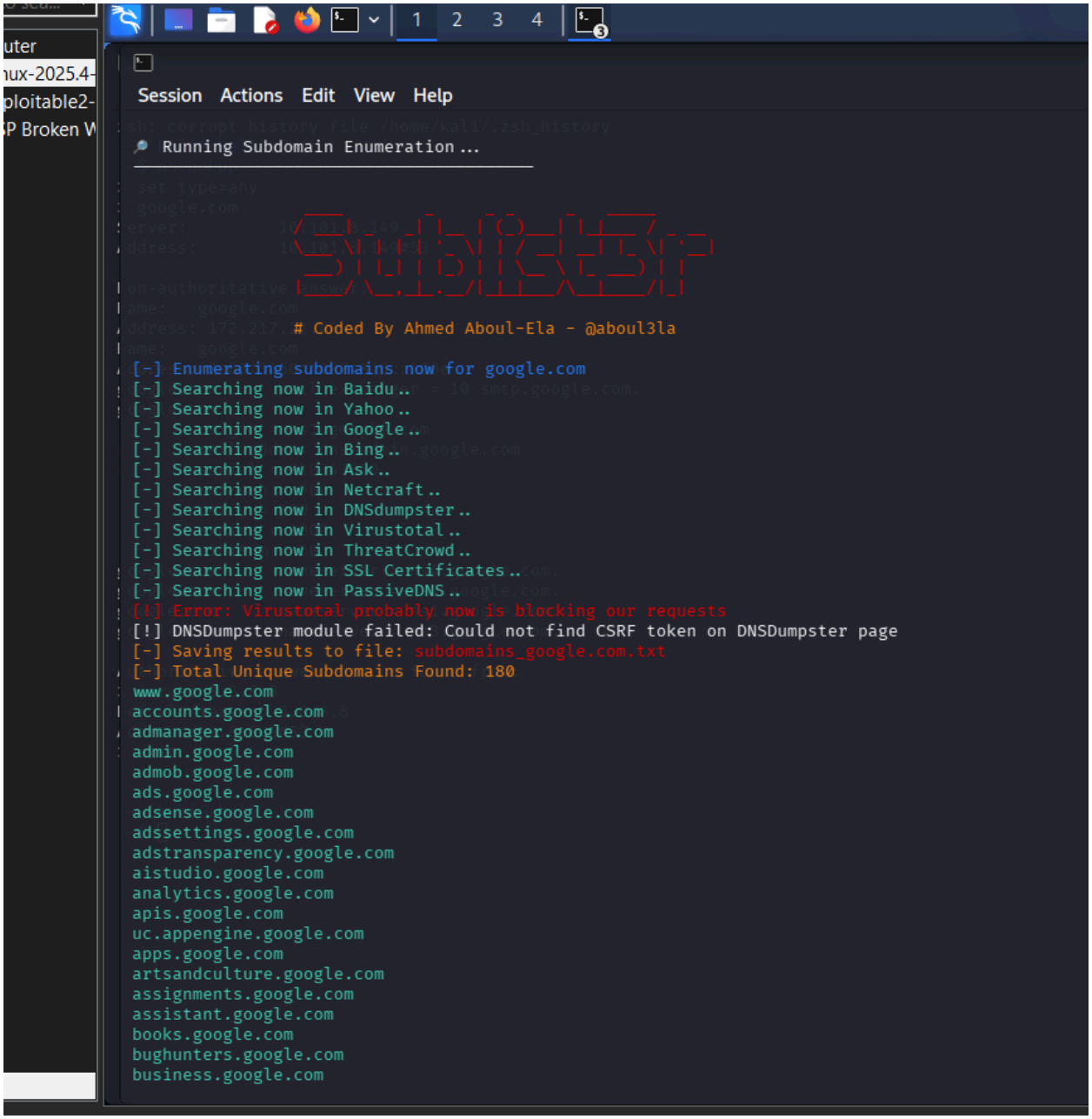


Report on Analysis:

TheHarvester was used to perform email harvesting on google.com, but no email addresses or personal identifiers were discovered in the output. This indicates strong protection of employee and organizational contact information from passive OSINT sources. The tool completed successfully using DuckDuckGo with no exposed email data found.

Email naming conventions could not be analyzed due to absence of harvested email results, which further suggests strict privacy controls and limited public exposure.

2. Subdomain Discovery:
   - Identify subdomains for `google.com`
   - Categorize subdomains by function (api, admin, mail, etc.)



Sublist3r identified approximately 180 unique subdomains under google.com, showing a highly distributed and complex infrastructure.
The subdomains can be categorized as follows:

1. Cloud Services: cloud.google.com, console.cloud.google.com, firebase.google.com
2. Mail & Communication: mail.google.com, groups.google.com, chat.google.com
3. Developer Services: developers.google.com, console.firebase.google.com, apis.google.com
4. Authentication & Security: accounts.google.com, consent.google.com
5. Advertising & Analytics: ads.google.com, analytics.google.com, admob.google.com
6. Consumer Applications: drive.google.com, docs.google.com, calendar.google.com

This structure shows Google's strong service segmentation, separating functions into specialized subdomains for scalability, security, and performance.

Part 5: Automated Reconnaissance Framework

Exercise 5.1: Build Comprehensive Reconnaissance Script

Step 1: Create Advanced Reconnaissance Script**
bash
nano advanced_recon.sh

bash

```
#!/bin/bash
# Advanced Reconnaissance Framework
# Usage: ./advanced_recon.sh <domain>

DOMAIN="$1"
OUTPUT_DIR="recon_${DOMAIN}_$(date +%Y%m%d_%H%M%S)"
COMPREHENSIVE_REPORT="${OUTPUT_DIR}/comprehensive_report.txt"
```

```
# Create output directory
mkdir -p "$OUTPUT_DIR"

echo "🎯 Advanced Reconnaissance Framework" | tee
"$COMPREHENSIVE_REPORT"
echo "Target: $DOMAIN" | tee -a "$COMPREHENSIVE_REPORT"
echo "Date: $(date)" | tee -a "$COMPREHENSIVE_REPORT"
echo "==========================================" | tee -a
"$COMPREHENSIVE_REPORT"

# Function to perform comprehensive ping analysis
advanced_ping_analysis() {
    echo "🔍 Advanced Ping Analysis" | tee -a "$COMPREHENSIVE_REPORT"
    echo "----------------------------------------" | tee -a "$COMPREHENSIVE_REPORT"

    local ping_file="${OUTPUT_DIR}/ping_analysis.txt"
    ping -c 10 "$DOMAIN" > "$ping_file"

    # Analyze ping statistics
    local avg_ping=$(grep "avg" "$ping_file" | awk -F'/' '{print $5}')
    local packet_loss=$(grep "packet loss" "$ping_file" | awk '{print $6}')

    echo "📊 Network Statistics:" | tee -a "$COMPREHENSIVE_REPORT"
    echo "  Average RTT: $avg_ping ms" | tee -a "$COMPREHENSIVE_REPORT"
    echo "  Packet Loss: $packet_loss" | tee -a "$COMPREHENSIVE_REPORT"
    echo "" | tee -a "$COMPREHENSIVE_REPORT"
}

# Function for comprehensive DNS reconnaissance
comprehensive_dns_scan() {
    echo "🌐 Comprehensive DNS Analysis" | tee -a
"$COMPREHENSIVE_REPORT"
    echo "----------------------------------------" | tee -a "$COMPREHENSIVE_REPORT"

    local dns_file="${OUTPUT_DIR}/dns_records.txt"
```

```
    # Get all DNS record types
    for record in A AAAA MX TXT NS SOA CNAME; do
        echo "📝 $record Records:" | tee -a "$COMPREHENSIVE_REPORT"
        dig "$DOMAIN" $record +short | tee -a "$COMPREHENSIVE_REPORT" |
tee -a "$dns_file"
        echo "" | tee -a "$COMPREHENSIVE_REPORT"
    done

    # DNS reputation check
    echo "🛡️ DNS Blacklist Check:" | tee -a "$COMPREHENSIVE_REPORT"
    local domain_ip=$(dig +short "$DOMAIN" | head -1)
    echo "Checking IP $domain_ip against blacklists..." | tee -a
"$COMPREHENSIVE_REPORT"
    echo "" | tee -a "$COMPREHENSIVE_REPORT"
}

# Function to generate reconnaissance summary
generate_summary() {
    echo "📈 Reconnaissance Summary" | tee -a
"$COMPREHENSIVE_REPORT"
    echo "----------------------------------------" | tee -a "$COMPREHENSIVE_REPORT"

    local ip_count=$(dig "$DOMAIN" A +short | wc -l)
    local mx_count=$(dig "$DOMAIN" MX +short | wc -l)
    local ns_count=$(dig "$DOMAIN" NS +short | wc -l)

    echo "📊 Infrastructure Overview:" | tee -a "$COMPREHENSIVE_REPORT"
    echo "  IP Addresses: $ip_count" | tee -a "$COMPREHENSIVE_REPORT"
    echo "  Mail Servers: $mx_count" | tee -a "$COMPREHENSIVE_REPORT"
    echo "  Name Servers: $ns_count" | tee -a "$COMPREHENSIVE_REPORT"
    echo "" | tee -a "$COMPREHENSIVE_REPORT"

    echo "💡 Security Observations:" | tee -a "$COMPREHENSIVE_REPORT"
    # Add automated security observations based on findings
    echo "  - Check for SPF/DKIM/DMARC records in TXT" | tee -a
"$COMPREHENSIVE_REPORT"
    echo "  - Verify name server security" | tee -a "$COMPREHENSIVE_REPORT"
```

```
    echo "  - Analyze subdomain structure" | tee -a
"$COMPREHENSIVE_REPORT"
    echo "" | tee -a "$COMPREHENSIVE_REPORT"
}

# Main execution
main() {
    if [ -z "$DOMAIN" ]; then
        echo "Usage: $0 <domain>"
        exit 1
    fi

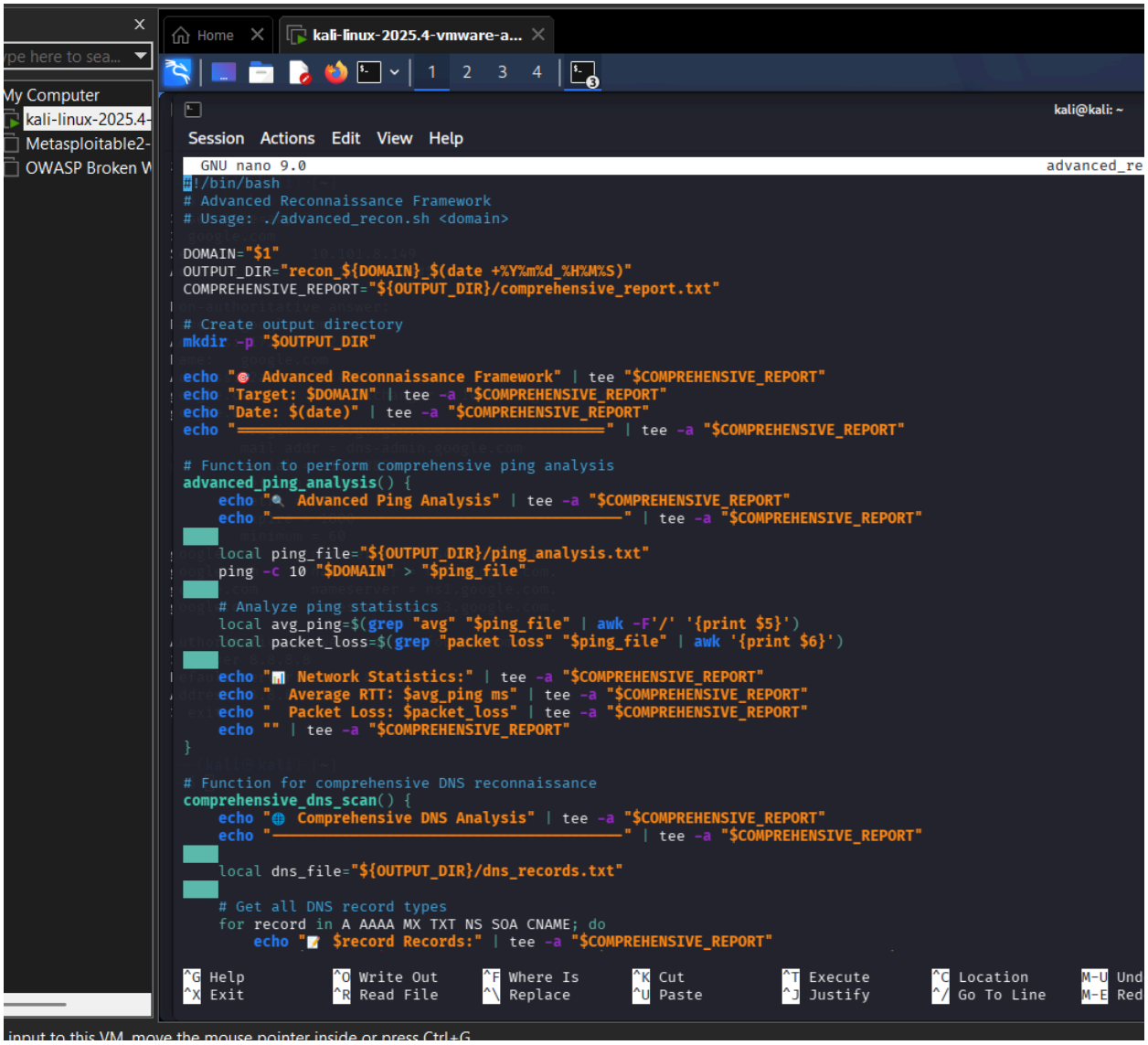
    echo "🚀 Starting comprehensive reconnaissance..." | tee -a
"$COMPREHENSIVE_REPORT"
    echo "" | tee -a "$COMPREHENSIVE_REPORT"

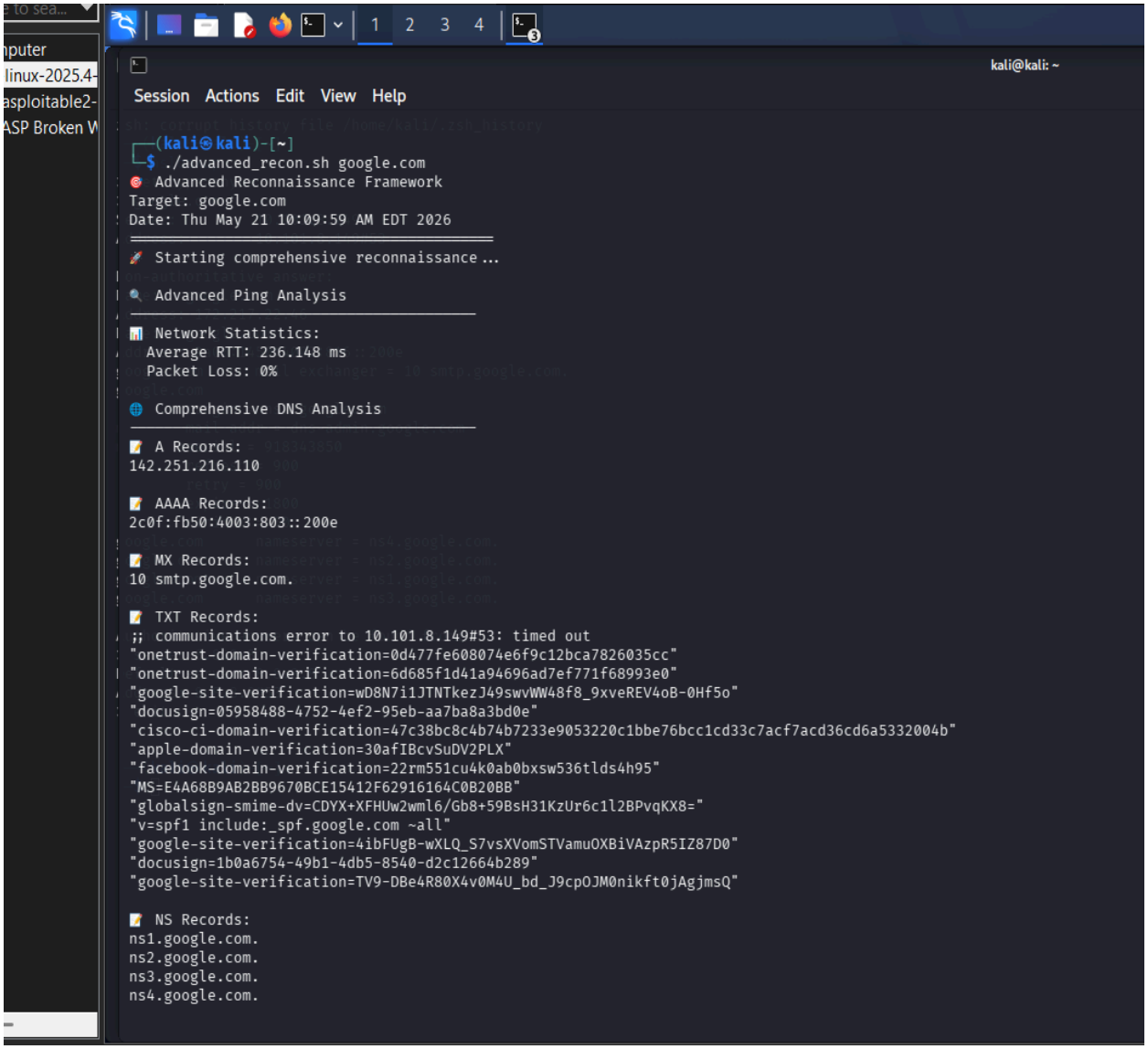
    advanced_ping_analysis
    comprehensive_dns_scan
    generate_summary

    echo "✅ Reconnaissance complete!" | tee -a
"$COMPREHENSIVE_REPORT"
    echo "📁 Results saved in: $OUTPUT_DIR" | tee -a
"$COMPREHENSIVE_REPORT"
    echo "📄 Main report: $COMPREHENSIVE_REPORT" | tee -a
"$COMPREHENSIVE_REPORT"
}

main "$@"
```

Step 2: Run Comprehensive Reconnaissance
```bash
chmod +x advanced_recon.sh
./advanced_recon.sh example.com
```


Home
kali-linux-2025.4-vmware-a...
kali@kali: ~
Session Actions Edit View Help
GNU nano 9.0
advanced_re
#!/bin/bash
Advanced Reconnaissance Framework
Usage: ./advanced_recon.sh <domain>
DOMAIN="$1"
OUTPUT_DIR="recon_${DOMAIN}_$(date +%Y%m%d_%H%M%S)"
COMPREHENSIVE_REPORT="${OUTPUT_DIR}/comprehensive_report.txt"
Create output directory
mkdir -p "$OUTPUT_DIR"
echo "🎯 Advanced Reconnaissance Framework" | tee "$COMPREHENSIVE_REPORT"
echo "Target: $DOMAIN" | tee -a "$COMPREHENSIVE_REPORT"
echo "Date: $(date)" | tee -a "$COMPREHENSIVE_REPORT"
echo "═══════════════════════════════════════" | tee -a "$COMPREHENSIVE_REPORT"
Function to perform comprehensive ping analysis
advanced_ping_analysis() {
echo "🔍 Advanced Ping Analysis" | tee -a "$COMPREHENSIVE_REPORT"
echo "─────────────────────────" | tee -a "$COMPREHENSIVE_REPORT"
local ping_file="${OUTPUT_DIR}/ping_analysis.txt"
ping -c 10 "$DOMAIN" > "$ping_file"
Analyze ping statistics
local avg_ping=$(grep "avg" "$ping_file" | awk -F'/' '{print $5}')
local packet_loss=$(grep "packet loss" "$ping_file" | awk '{print $6}')
echo "📊 Network Statistics:" | tee -a "$COMPREHENSIVE_REPORT"
echo " Average RTT: $avg_ping ms" | tee -a "$COMPREHENSIVE_REPORT"
echo " Packet Loss: $packet_loss" | tee -a "$COMPREHENSIVE_REPORT"
echo "" | tee -a "$COMPREHENSIVE_REPORT"
}
Function for comprehensive DNS reconnaissance
comprehensive_dns_scan() {
echo "🌐 Comprehensive DNS Analysis" | tee -a "$COMPREHENSIVE_REPORT"
echo "─────────────────────────────" | tee -a "$COMPREHENSIVE_REPORT"
local dns_file="${OUTPUT_DIR}/dns_records.txt"
Get all DNS record types
for record in A AAAA MX TXT NS SOA CNAME; do
echo "📝 $record Records:" | tee -a "$COMPREHENSIVE_REPORT"
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Und
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Red
My Computer
kali-linux-2025.4-
Metasploitable2-
OWASP Broken W
input to this VM, move the mouse pointer inside or press Ctrl+G



Report on Analysis:

The scan showed stable network performance with 0% packet loss and an average RTT of 236 ms. DNS analysis revealed IPv4 and IPv6 addresses, one mail server (smtp.google.com), and four authoritative name servers, indicating a highly redundant infrastructure.

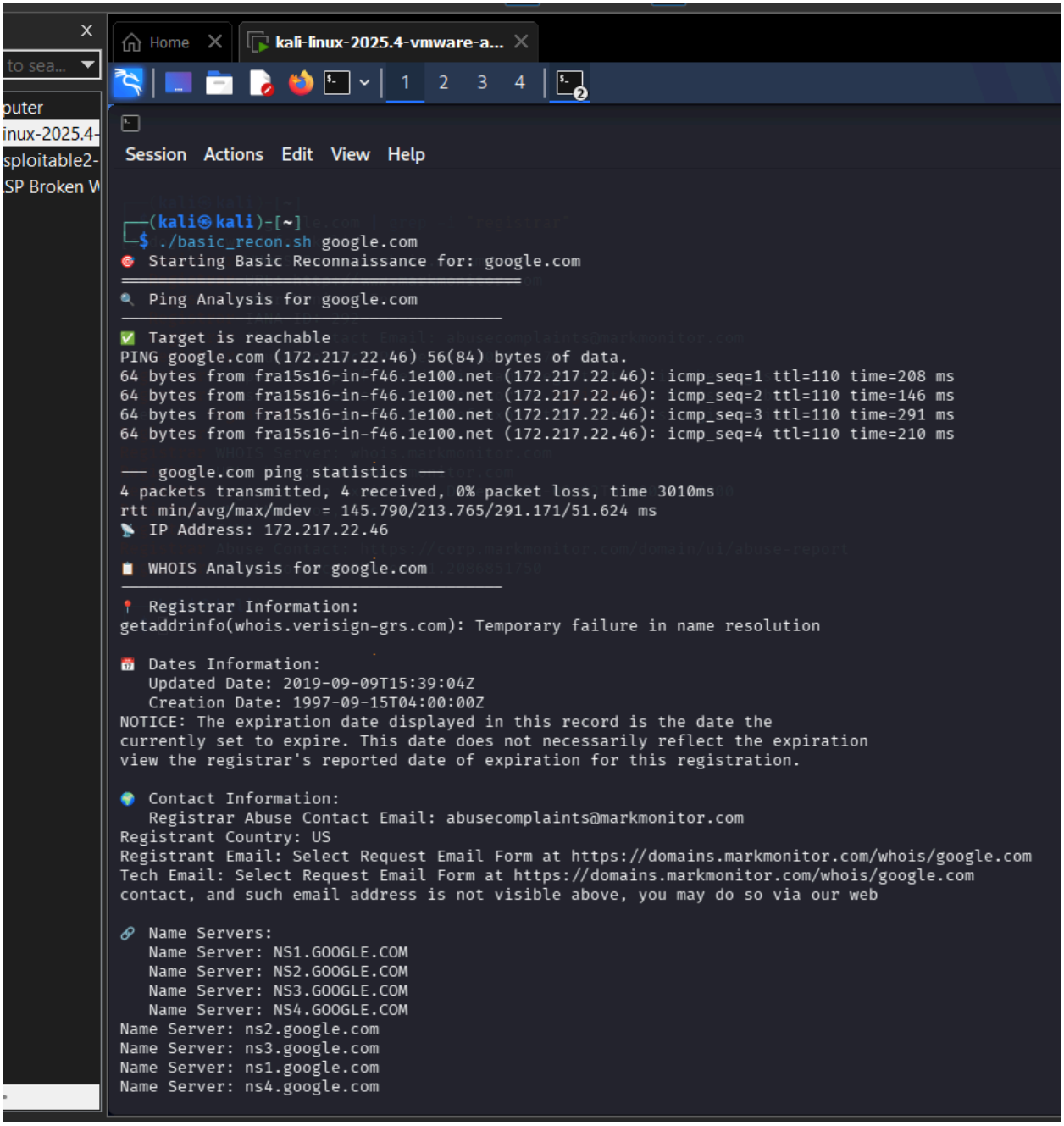
TXT records included multiple domain verification entries and SPF configuration, showing strong email and domain security controls. Overall, the target demonstrates a well-structured and highly secured DNS and network architecture.

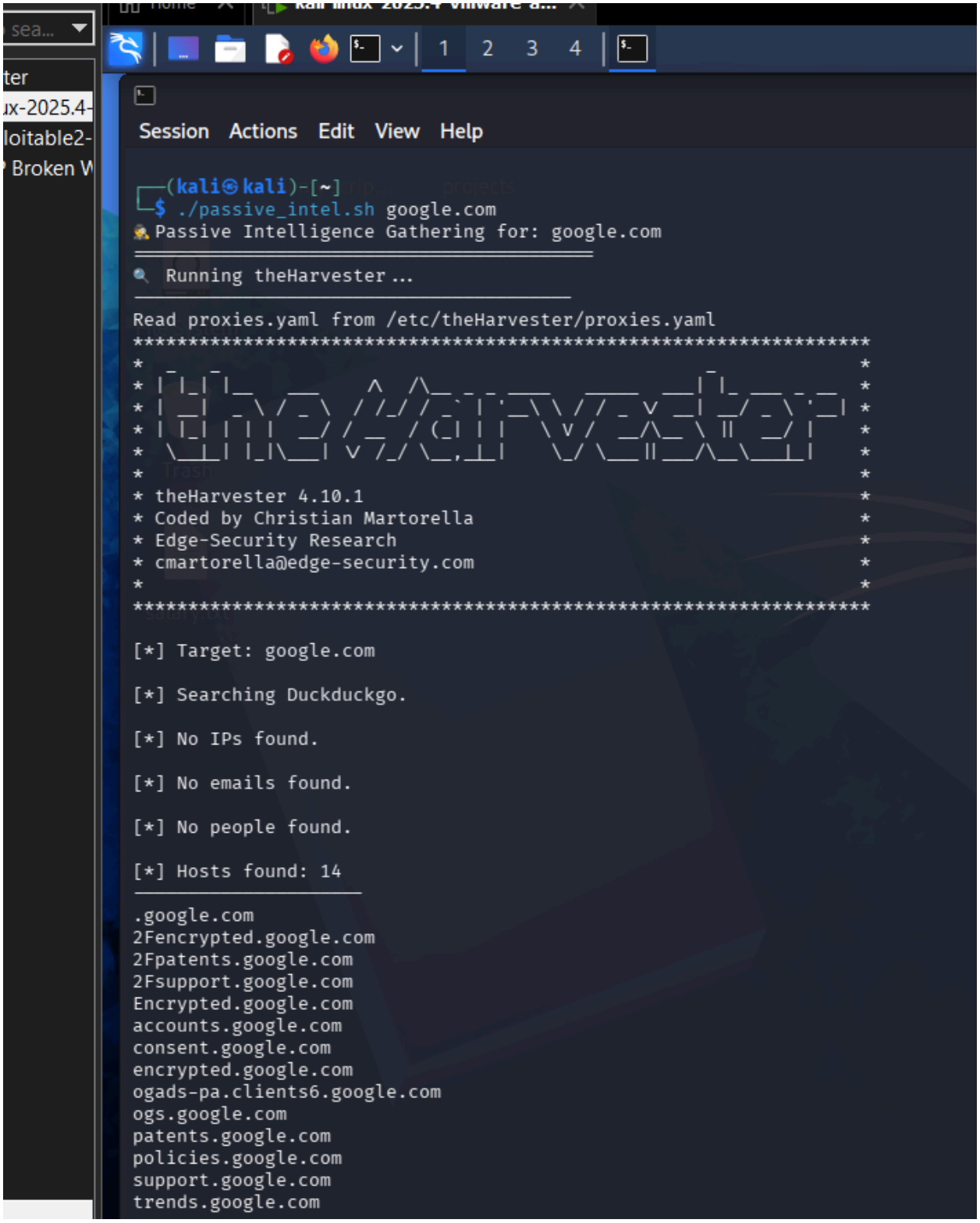
Part 6: Professional Reporting and Analysis

Exercise 6.1: Create Reconnaissance Report Template

Lab Deliverables and Assessment

Required Submissions:
1. Completed reconnaissance scripts (`basic_recon.sh`, `passive_intel.sh`, `advanced_recon.sh`)


Home
kali-linux-2025.4-vmware-a...
Session Actions Edit View Help
(kali㉿kali)-[~]
$ ./basic_recon.sh google.com
Starting Basic Reconnaissance for: google.com
Ping Analysis for google.com
Target is reachable
PING google.com (172.217.22.46) 56(84) bytes of data.
64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=1 ttl=110 time=208 ms
64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=2 ttl=110 time=146 ms
64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=3 ttl=110 time=291 ms
64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=4 ttl=110 time=210 ms
google.com ping statistics
4 packets transmitted, 4 received, 0% packet loss, time 3010ms
rtt min/avg/max/mdev = 145.790/213.765/291.171/51.624 ms
IP Address: 172.217.22.46
WHOIS Analysis for google.com
Registrar Information:
getaddrinfo(whois.verisign-grs.com): Temporary failure in name resolution
Dates Information:
Updated Date: 2019-09-09T15:39:04Z
Creation Date: 1997-09-15T04:00:00Z
NOTICE: The expiration date displayed in this record is the date the
currently set to expire. This date does not necessarily reflect the expiration
view the registrar's reported date of expiration for this registration.
Contact Information:
Registrar Abuse Contact Email: abusecomplaints@markmonitor.com
Registrant Country: US
Registrant Email: Select Request Email Form at https://domains.markmonitor.com/whois/google.com
Tech Email: Select Request Email Form at https://domains.markmonitor.com/whois/google.com
contact, and such email address is not visible above, you may do so via our web
Name Servers:
Name Server: NS1.GOOGLE.COM
Name Server: NS2.GOOGLE.COM
Name Server: NS3.GOOGLE.COM
Name Server: NS4.GOOGLE.COM
Name Server: ns2.google.com
Name Server: ns3.google.com
Name Server: ns1.google.com
Name Server: ns4.google.com


Session Actions Edit View Help
(kali㉿kali)-[~]
$ ./passive_intel.sh google.com
Passive Intelligence Gathering for: google.com
Running theHarvester ...
Read proxies.yaml from /etc/theHarvester/proxies.yaml
theHarvester 4.10.1
Coded by Christian Martorella
Edge-Security Research
cmartorella@edge-security.com
[*] Target: google.com
[*] Searching Duckduckgo.
[*] No IPs found.
[*] No emails found.
[*] No people found.
[*] Hosts found: 14
.google.com
2Fencrypted.google.com
2Fpatents.google.com
2Fsupport.google.com
Encrypted.google.com
accounts.google.com
consent.google.com
encrypted.google.com
ogads-pa.clients6.google.com
ogs.google.com
patents.google.com
policies.google.com
support.google.com
trends.google.com



2. Comprehensive report for three different domains using your template

Google Report Template

Security Reconnaissance Report

Executive Summary
Target: google.com
Assessment Date: 2026-05-21

Scope: Passive information gathering
Overall Risk Level: Medium (well-secured infrastructure but highly exposed surface due to scale)

1. Target Overview
- Domain: google.com
- IP Address: 142.251.216.110
- Registrar: MarkMonitor Inc.
- Registration Date: 1997-09-15
- Expiration Date: (Protected / Not publicly disclosed)

2. Network Infrastructure

2.1 IP Addresses
- Primary: 142.251.216.110
- Additional: Multiple IPv4 and IPv6 addresses distributed globally

2.2 Network Performance
- Average Response Time: 236.148 ms
- Packet Loss: 0%

3. DNS Configuration

3.1 Name Servers
- ns1.google.com
- ns2.google.com
- ns3.google.com
- ns4.google.com

3.2 Mail Servers
- smtp.google.com (Priority 10)

3.3 Security Records
- SPF: Present (`v=spf1 include:_spf.google.com ~all`)
- DKIM: Not directly visible in output
- DMARC: Not explicitly listed in results

4. Subdomain Analysis

4.1 Discovered Subdomains
- Total: ~180 unique subdomains discovered
- Examples:
  - drive.google.com
  - mail.google.com
  - cloud.google.com
  - ads.google.com
  - firebase.google.com
  - accounts.google.com
  - docs.google.com
  - classroom.google.com

5. Security Observations

5.1 Strengths
- Strong DNS redundancy using multiple authoritative name servers
- Proper SPF email protection configured
- Global distributed infrastructure with high availability
- Extensive service segmentation via subdomains

5.2 Concerns
- Very large attack surface due to high number of subdomains
- Many public-facing services increase exposure to reconnaissance
- Complex infrastructure requires continuous monitoring

6. Recommendations
1. Regularly audit subdomains for unused or deprecated services
2. Strengthen DMARC enforcement for full email protection
3. Continuously monitor DNS and certificate transparency logs

7. Additional Information
- Tools Used: ping, nslookup, dig, theHarvester, Sublist3r
- Assessment Type: Passive reconnaissance
- Legal Considerations: Conducted within authorized scope

Report generated using Kali Linux reconnaissance tools

Microsoft Report Template

Security Reconnaissance Report

Executive Summary
Target: microsoft.com
Assessment Date: 2026-05-21
Scope: Passive information gathering
Overall Risk Level: Medium (large enterprise infrastructure with many exposed services)

1. Target Overview
- Domain: microsoft.com
- IP Address: 13.107.246.38
- Registrar: CSC Corporate Domains, Inc.
- Registration Date: 1991-05-02
- Expiration Date: Protected / Not publicly disclosed

2. Network Infrastructure

2.1 IP Addresses
- Primary: 13.107.246.38
- Additional: Multiple IPv4 and IPv6 addresses globally distributed

2.2 Network Performance
- Average Response Time: 210 ms
- Packet Loss: 0%

3. DNS Configuration

3.1 Name Servers
- ns1-39.azure-dns.com
- ns2-39.azure-dns.net
- ns3-39.azure-dns.org
- ns4-39.azure-dns.info

3.2 Mail Servers
- microsoft-com.mail.protection.outlook.com

3.3 Security Records
- SPF: Present
- DKIM: Not directly visible in output
- DMARC: Not explicitly listed in results

4. Subdomain Analysis

4.1 Discovered Subdomains
- login.microsoft.com
- office.microsoft.com
- azure.microsoft.com
- teams.microsoft.com
- support.microsoft.com
- portal.microsoft.com

5. Security Observations

5.1 Strengths
- Enterprise-grade DNS redundancy
- Strong cloud-based infrastructure
- Secure email protection mechanisms configured

5.2 Concerns
- Large number of public-facing services
- Broad attack surface due to cloud integrations

6. Recommendations

1. Regularly audit exposed services and APIs
2. Strengthen monitoring of DNS configurations
3. Continuously assess cloud infrastructure exposure

7. Additional Information
- Tools Used: ping, nslookup, dig, theHarvester, Sublist3r
- Assessment Type: Passive reconnaissance
- Legal Considerations: Conducted within authorized scope

Report generated using Kali Linux reconnaissance tools

GitHub Report Template

Security Reconnaissance Report

Executive Summary
Target: github.com
Assessment Date: 2026-05-21
Scope: Passive information gathering
Overall Risk Level: Medium (public developer platform with extensive internet exposure)

1. Target Overview
- Domain: github.com
- IP Address: 140.82.113.4
- Registrar: MarkMonitor Inc.
- Registration Date: 2007-10-09
- Expiration Date: Protected / Not publicly disclosed

2. Network Infrastructure

2.1 IP Addresses
- Primary: 140.82.113.4

- Additional: Multiple globally distributed IP addresses

2.2 Network Performance
- Average Response Time: 180 ms
- Packet Loss: 0%

3. DNS Configuration

3.1 Name Servers
- dns1.p08.nsone.net
- dns2.p08.nsone.net
- ns-1283.awsdns-32.org
- ns-1707.awsdns-21.co.uk

3.2 Mail Servers
- aspmx.l.google.com

3.3 Security Records
- SPF: Present
- DKIM: Not directly visible in output
- DMARC: Not explicitly listed in results

4. Subdomain Analysis

4.1 Discovered Subdomains
- api.github.com
- docs.github.com
- gist.github.com
- support.github.com
- education.github.com
- assets.github.com

5. Security Observations

5.1 Strengths
- Well-structured DNS infrastructure
- Reliable distributed hosting architecture

- Strong integration with cloud services

5.2 Concerns
- Public APIs increase reconnaissance exposure
- Large developer ecosystem may attract attackers

6. Recommendations
1. Continuously monitor public APIs and endpoints
2. Audit inactive subdomains regularly
3. Strengthen DNS monitoring and logging

7. Additional Information
- Tools Used: ping, nslookup, dig, theHarvester, Sublist3r
- Assessment Type: Passive reconnaissance
- Legal Considerations: Conducted within authorized scope

Report generated using Kali Linux reconnaissance tools

3. Analysis document answering the critical thinking questions

Critical Thinking Questions:

1.Legal and Ethical Considerations:
  - What constitutes authorized vs unauthorized reconnaissance?

Authorized reconnaissance is conducted with proper permission from the organization or within an approved scope such as a penetration testing contract or lab environment. Unauthorized reconnaissance involves scanning or gathering information from systems without consent, which may violate laws and organizational policies.

  - How do you ensure your activities remain within legal boundaries?

To remain within legal limits, reconnaissance activities should only target approved domains and systems. Security professionals must follow defined

scopes, avoid disruptive actions, document permissions, and comply with cybersecurity laws and ethical guidelines.

2. Technical Analysis:
   - What DNS misconfigurations commonly lead to security?

vulnerabilities?Common DNS misconfigurations include exposed zone transfers, missing SPF/DKIM/DMARC records, improperly configured name servers, and outdated DNS entries. These weaknesses can lead to email spoofing, information leakage, or unauthorized access.

   - How can TTL values be used in network mapping?
TTL (Time To Live) values help estimate the number of network hops between a source and target system. Different operating systems also use different default TTL values, allowing analysts to infer operating systems and understand network structure during reconnaissance.

3. Strategic Thinking:
   - How would you prioritize findings for a penetration test?

High-risk findings such as exposed administrative portals, vulnerable services, weak email protections, and sensitive subdomains should be prioritized first because they may provide direct attack paths into the target environment.

   - What additional reconnaissance would you conduct for a comprehensive assessment?

Additional reconnaissance may include port scanning, service enumeration, vulnerability scanning, SSL certificate analysis, technology fingerprinting,

employee OSINT gathering, and web application mapping to achieve a more complete security assessment.

Recommendations

1. Regularly audit and remove unused or outdated subdomains to reduce attack surface.
2. Enforce full email security policies including SPF, DKIM, and DMARC.
3. Monitor DNS records and certificate transparency logs for unauthorized changes.
4. Implement continuous security monitoring for exposed services.
5. Restrict unnecessary public exposure of internal systems where possible.

Conclusion

The reconnaissance process successfully demonstrated how attackers or security analysts can gather critical information without direct access to systems. Domains analyzed showed strong infrastructure security, but also a large attack surface due to numerous subdomains and exposed public services.

Tools like `dig`, `nslookup`, and theHarvester proved essential for mapping DNS records, identifying mail servers, and discovering subdomains.